Modern AI Foundations

An Accelerated Introduction to AI:
From Classical Machine Learning
to Large Language Models



An accelerated, code-first guide to the ideas behind modern artificial intelligence: the mathematics that powers it, the classical machine-learning engine at its heart, and the deep-learning revolution that brought us transformers and large language models — with curated sources for every step of the journey.

Prerequisites: Intermediate Python • Basic NumPy & Pandas • Calculus & Linear Algebra Basics

Course Design Team

Version 1.1 • September 2025 • License: CC BY 4.0

Repository: https://github.com/DanielBerns/AI-course

Contents

About this course 2

Prerequisites and required background 2

1 Introduction and the AI Landscape 4
- 1.1 Defining intelligence: weak vs. strong AI . . . 4
- 1.2 From expert systems to data-driven AI . . . 4
- 1.3 The modern stack: compute, cloud, and data pipelines . . . 5
- 1.4 Ethics, safety, and security: the non-optional module . . . 6

2 Mathematics and Data Foundations 9
- 2.1 Linear algebra: the language of data . . . 9
- 2.2 Probability and statistics: the language of uncertainty . . . 10
- 2.3 Optimisation: how models learn . . . 11

3 Classical Machine Learning: The Core Engine 14
- 3.1 Supervised learning . . . 14
- 3.2 Unsupervised learning . . . 21
- 3.3 Reinforcement learning . . . 22
- 3.4 Modern evaluation methodology & diagnostics . . . 23

4 Transition to Deep Learning and Modern AI 25
- 4.1 Neural networks: perceptrons, activations, backpropagation . . . 25
- 4.2 Representation learning: from manual engineering to latent spaces . . . 27
- 4.3 Architectures: CNNs, RNNs, and transformers . . . 30
- 4.4 Self-supervised learning: unifying supervised and unsupervised paradigms . . . 31
- 4.5 Generative AI & Foundation Models: Adaptation and Inference . . . 34

Where to go next 37

Practical Coding Projects Series 38

Appendix: Answer Key for Check Your Understanding 39

Glossary 41

References 42

About this course

This document develops a four-module accelerated overview of modern artificial intelligence for technically capable students. It assumes basic background summarised in the prerequisite matrix on the next page: comfortable **Python**, basic **NumPy & Pandas**, and **elementary calculus and linear algebra**. Everything else — from eigenvectors to attention — is built from scratch, in context, with citations to the primary literature and standard textbooks.

The four modules are deliberately sequential, as the roadmap below shows: the landscape and its ethics (*Module 1*), the mathematical foundations (*Module 2*), the classical machine-learning engine that remains highly competitive for tabular workloads (*Module 3*), and the deep learning and generative-AI frontier (*Module 4*). All code snippets assume Python 3.9+ with `numpy`, `pandas`, `scikit-learn`, and (for Module 4) `transformers` installed.

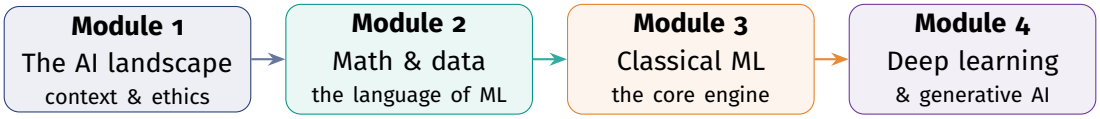


Figure 1. The learning path: each module builds directly on the one before it.

Throughout the guide you will meet the following recurring callout boxes:

Box	What it gives you
Module — at a glance	Focus, key concepts, and learning goals for each module.
Key concept	Definitions and intuitions worth memorising.
Hands-on with Python	Short, runnable code tied to the prerequisite skills.
Ethics & safety spotlight	Responsible-AI notes and open questions.
Check your understanding	A quick self-test before moving on.
Sources & further reading	Curated references backing each module.

Table 1. The callout boxes used throughout this guide.

Prerequisites and required background

The table below is the assumed starting point. None of the rows requires expert mastery; the goal of each is narrow and practical.

Skill	Required level	Primary use case
Python programming	Intermediate	Implementing algorithms, data manipulation
NumPy & Pandas	Basic	Matrix operations and data cleaning
Calculus & Linear Algebra	Basic	Derivatives, partials, vectors, and matrices

Table 2. Prerequisite matrix: the background assumed by this guide.

Python (intermediate). You should be comfortable writing functions, loops, list comprehensions, and simple classes, and know your way around a virtual environment. The course is deliberately code-first: every mathematical idea is paired with a few lines of Python. A good refresher, free to read online, is VanderPlas (2016).

Table 3. Notation reference table.

Symbol / Notation	Description and Typical Context
$\mathbf{x}, \mathbf{y}, \mathbf{z}$	Bold lowercase letters denote column vectors (e.g., sample features $\mathbf{x} \in \mathbb{R}^n$).
X, W, A	Uppercase letters denote matrices (e.g., dataset matrix $X \in \mathbb{R}^{m \times n}$, weights W).
$\boldsymbol{\theta}, \mathbf{w}, \boldsymbol{\beta}$	Model parameter vectors (e.g., regression weights $\boldsymbol{\beta}$, neural net parameters $\boldsymbol{\theta}$).
$\mathcal{L}(\hat{y}, y),\ J(\boldsymbol{\theta})$	Loss function for a single instance ($\mathcal{L}$) vs. cost function averaged over data (J).
$\nabla_{\boldsymbol{\theta}} J$	Gradient vector of partial derivatives with respect to parameters $\boldsymbol{\theta}$.
Σ vs. $\sum$	Uppercase sigma Σ denotes a Covariance Matrix; summation operator $\sum_{i=1}^{m}$ denotes summing a series.
$\odot$	Hadamard (element-wise) vector or matrix product.
$\phi(z), \phi'(z)$	Activation function and its scalar derivative.

NumPy & Pandas (basic). You should be able to create arrays, slice them, and compute with them (`A @ B` for matrix products), and load a CSV into a DataFrame to clean and summarise it (Harris et al. 2020; McKinney 2010). The practice boxes assume exactly this level and no more.

Calculus (basic). You need the derivative as a slope, the power rule, and the chain rule. That is genuinely all — gradients, the engine of all learning in this course, are just derivatives collected into a vector.

The two calculus ideas you actually need

- **A derivative is a local slope:** $f'(x) \approx \dfrac{f(x+h) - f(x)}{h}$ — it tells you which direction is *downhill* and how steeply.
- **The chain rule composes slopes:** $\dfrac{d}{dx} f(g(x)) = f'(g(x))\, g'(x)$.

Backpropagation (Module 4) — the efficient gradient-computation algorithm used to train modern neural networks — is nothing more than this second rule applied systematically, layer by layer.

Self-check before starting.

- Can you write a Python function that returns the mean and standard deviation of a list of numbers?
- Given two NumPy arrays, can you compute their dot product and the shape of `A @ B`?
- Can you differentiate $f(w) = w^2$ and say what $f'(3)$ means geometrically?

If any answer is “not yet”, spend a week with VanderPlas (2016) and any standard calculus refresher; then return — everything else is built for you from here.

1 Introduction and the AI Landscape

Module 1 — at a glance (Estimated Study: 4 hours • Level: Beginner)

Focus. What artificial intelligence is (and is not), how the field evolved from hand-written rules to learning from data, and the infrastructure and ethical questions that shape modern practice.

Key concepts. Defining intelligence; weak vs. strong AI; expert systems vs. data-driven AI; GPUs, TPUs, cloud ecosystems and data pipelines; bias, alignment, copyright and environmental impact.

After this module you will be able to explain the difference between narrow and general AI, describe the paradigm shift from rules to data, sketch an end-to-end machine-learning pipeline, and name the main ethical questions attached to deploying an AI system.

1.1 Defining intelligence: weak vs. strong AI

Before we can build artificial intelligence we have to say what we mean by *intelligence*. Russell and Norvig (2021) organise the many definitions along two axes: are we concerned with *thought processes* or with *behaviour*, and should the benchmark be *human-like* performance or *rationality* — acting so as to achieve the best expected outcome? Alan Turing sidestepped the philosophy in 1950 with an operational test: if a machine's written answers are indistinguishable from a human's, is it not, for practical purposes, intelligent? (Turing 1950). Modern AI research mostly aims at rational behaviour on well-defined tasks — exactly what the learning algorithms of Modules 2–4 deliver.

Weak (narrow) AI vs. strong AI

Weak / narrow AI (today)	Strong AI / AGI (hypothetical)
Excels at one well-defined task: spam filters, image classifiers, next-token prediction	Reasons, plans and learns across arbitrary domains, as a human can
Learns from task-specific data; capability does not transfer	Would transfer knowledge flexibly to novel problems
Every deployed system, including today's LLMs, is narrow in this sense	Unknown whether or when it is achievable; an active research and policy debate

A second classical debate is worth knowing. Philosopher John Searle's *Chinese room* argument holds that symbol manipulation alone is not understanding: a person following rules could answer questions in Chinese without knowing any Chinese (Searle 1980). The argument has returned with force in the era of large language models, because fluent output is cheap while grounded understanding is hard to verify — a theme we revisit in the ethics spotlight below.

1.2 From expert systems to data-driven AI

The term *artificial intelligence* was coined by John McCarthy in the 1955 proposal for the 1956 Dartmouth workshop. For roughly the next three decades the dominant paradigm was *symbolic*: human knowledge was hand-encoded as if–then rules. Expert systems such as MYCIN encoded

several hundred rules elicited from physicians and impressed clinicians in blind evaluations — yet only inside their narrow domain. Rules are brittle, and extracting them from experts (the *knowledge-acquisition bottleneck*) is slow and expensive.

Machine learning inverts the pipeline: instead of programming rules, we collect examples of the desired behaviour and let an algorithm discover the rules. In Mitchell (1997)'s classic formulation, a program *learns* from experience E with respect to task T and performance measure P if its performance at T, measured by P, improves with E. Deep learning pushes the inversion one step further by learning not only the rules but the *features* they operate on (LeCun et al. 2015). Three results announced the new era: AlexNet cut the ImageNet top-5 error rate from roughly 26% to 16% by training a deep network on two consumer GPUs (Krizhevsky et al. 2012); AlphaGo defeated the world Go champion using deep networks combined with search (Silver et al. 2016); and transformer language models (Module 4) turned plain text prediction into a general-purpose technology.

The paradigm shift: from rules to data

Classical programming: rules + data $\longrightarrow$ answers
Machine learning: data + answers $\longrightarrow$ rules (the model)

Instead of hand-crafting the logic, we specify an objective and let an algorithm search for parameters that meet it — which is precisely why the mathematics of Modules 2 and 3 matters so much.

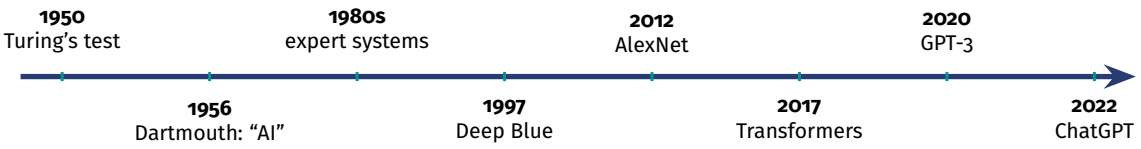


Figure 2. A selective history: from the founding of the field to the generative-AI era.

1.3 The modern stack: compute, cloud, and data pipelines

Compute: GPUs and TPUs. Training a modern model is, computationally, one thing: enormous numbers of matrix–vector products. A CPU optimises a handful of fast, sequential cores; a GPU packs thousands of simpler cores that perform these products in parallel. That is why Krizhevsky et al. (2012) could train a vision network that CPU-era attempts could not, and why hardware co-design now shapes chip roadmaps — Google's Tensor Processing Units, for instance, execute the tensor algebra of neural networks directly (Jouppi et al. 2017). Memory bandwidth is the other half of the story: modern accelerators are as much about feeding data as about computing with it.

Cloud ecosystems. The same hardware is available to anyone, rented by the hour: a laptop, a browser, and a credit card are enough to requisition a cluster of GPUs with a reproducible software image. This democratised experimentation — and concentrated an enormous amount of AI capability in a handful of providers.

Data engineering, dataset construction, and contamination. Models live inside a loop, not a vacuum (Figure 3). Practitioners spend most project time collecting, cleaning, deduplicating, and validating data (McKinney 2010). Modern AI development requires rigorous data hygiene:

- **Dataset Construction & Labeling Noise:** Human annotation introduces errors, ambiguities, and labeler bias. Clean labels are often more valuable than model architecture tweaks.
- **Train/Test Leakage & Contamination:** Pretraining datasets scraped at web scale frequently

suffer from *benchmark contamination* — test set questions leaking into training corpora, inflating evaluation scores artificially. Temporal leakage (using future data to predict past events) and group leakage (splitting samples from the same patient/user across train and test sets) yield invalid accuracy claims.

- **Data Augmentation & Synthetic Data:** Generating synthetic training examples or expanding datasets via domain-specific transforms mitigates data scarcity but risks amplifying generator biases.

Hands-on with Python

```python
import pandas as pd

df = pd.read_csv("orders.csv", parse_dates=["created"])
df = df.dropna(subset=["amount"])           # basic cleaning
df["revenue"] = df["amount"] * df["quantity"]
summary = df.groupby("region")["revenue"].agg(["count", "mean", "sum"])
print(summary)
```

Five lines of Pandas cover the ingest, clean, transform, aggregate loop that occupies most of a data scientist's day.

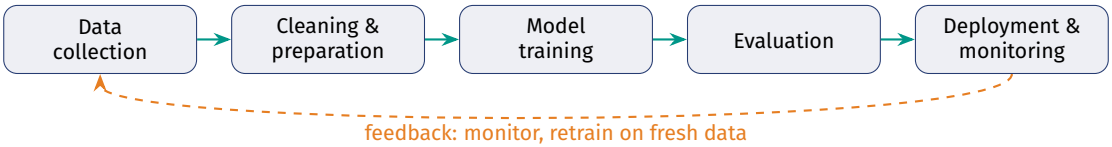


Figure 3. A modern data pipeline: the machine-learning lifecycle as a loop, not a line.

1.4 Ethics, safety, and security: the non-optional module

Technical capability has outpaced governance. Six essential dimensions should accompany every system you build:

Core dimensions of responsible AI and safety

Bias & Fairness. Models inherit patterns from training data. Commercial classifiers have shown severe demographic accuracy gaps (Buolamwini and Gebru 2018). Crucially, *simply stripping demographic features (e.g., race, gender) does NOT guarantee fairness*, because correlated proxy variables retain discriminatory signal. Furthermore, mathematical fairness criteria (such as Equalized Odds vs. Predictive Parity) frequently conflict under unequal base rates: **there is no single universally correct fairness metric**.

Privacy & Security. Training on sensitive records risks membership inference and model inversion attacks. Safeguards include Differential Privacy (ϵ, δ-guarantees) and federated aggregation. Additionally, deployment surfaces new threat vectors: *prompt injection*, data poisoning, and model extraction.

Alignment & Harm Mitigation. Objective misalignment causes reward hacking (Amodei et al. 2016; Christian 2020). Red teaming, safety guardrails, and preference optimization (Module 4) help steer foundation models toward helpful and safe execution.

Copyright & Provenance. Data scraping raises unresolved legal questions surrounding fair use, licensing, watermarking, and artist attribution.

Environment. Large-scale model training consumes significant energy and water (Strubell et al. 2019; Patterson et al. 2021). Mitigations include right-sizing architectures and using energy-efficient accelerators.

Automation Bias & Grounding. Bender et al. (2021) warned that LLMs are *stochastic parrots* reproducing surface-level statistics without grounded comprehension. Over-reliance (automation bias) leads users to accept unverified model claims.

A practitioner's checklist.

- Define the task, population served, and failure modes *before* modeling.
- Audit training data for representation gaps, proxy correlations, and leakage.
- Evaluate metrics across demographic sub-groups; balance cost trade-offs.
- Document limitations via Model Cards, Datasheets, and clear system prompts.
- Establish red teaming, active monitoring, and human-in-the-loop oversight.

Check your understanding

1. A chess engine, a spam filter, and a claimed "fully autonomous reasoning" startup demo: classify each as weak or strong AI, and justify the label.
2. Why did GPUs unlock deep learning at scale, whereas CPUs could not supply the necessary arithmetic throughput? Answer in terms of parallel hardware architecture.
3. Your model screens job applications. Name two concrete harms and one mitigation for each.

Sources & further reading

- S. J. Russell and P. Norvig (2021). *Artificial Intelligence: A Modern Approach*. 4th ed. Hoboken, NJ: Pearson
- A. M. Turing (1950). "Computing Machinery and Intelligence". In: *Mind* 59.236, pp. 433–460
- J. R. Searle (1980). "Minds, Brains, and Programs". In: *Behavioral and Brain Sciences* 3.3, pp. 417–424
- Y. LeCun et al. (2015). "Deep Learning". In: *Nature* 521.7553, pp. 436–444
- A. Krizhevsky et al. (2012). "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems 25*, pp. 1097–1105
- D. Silver et al. (2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587, pp. 484–489
- N. P. Jouppi et al. (2017). "In-Datacenter Performance Analysis of a Tensor Processing Unit". In: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–12
- J. Buolamwini and T. Gebru (2018). "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification". In: *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*, pp. 77–91
- D. Amodei et al. (2016). "Concrete Problems in AI Safety". In: *arXiv preprint arXiv:1606.06565*
- B. Christian (2020). *The Alignment Problem: Machine Learning and Human Values*. New York: W. W. Norton & Company
- E. Strubell et al. (2019). "Energy and Policy Considerations for Deep Learning in NLP". in: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3645–3650
- D. Patterson et al. (2021). "Carbon Emissions and Large Neural Network Training". In: *Advances in Neural Information Processing Systems 34*, pp. 13470–13481
- E. M. Bender et al. (2021). "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*

(FAccT), pp. 610–623

2 Mathematics and Data Foundations

Module 2 — at a glance (Estimated Study: 8 hours • Level: Intermediate)

Focus. The small mathematical toolkit that quietly powers everything else in the course: linear algebra to *represent* data, probability to *quantify* uncertainty, and optimisation to *learn* from mistakes.

Key concepts. Vectors, matrices, eigenvalues and dimensionality reduction; Bayes' theorem, distributions and hypothesis testing; loss and cost functions, gradients, and gradient descent.

After this module you will be able to describe any dataset as linear-algebra objects, reason correctly about conditional probabilities, and explain — and implement — gradient descent from scratch.

Mathematics here is a working tool, not an exam subject. Each concept is introduced the moment it earns its keep, and each is paired with NumPy code that uses it.

2.1 Linear algebra: the language of data

Machine-learning data lives in vectors, matrices, and tensors. A house becomes a vector of measurements $\mathbf{x} = [\text{area}, \text{rooms}, \text{age}]$; a dataset of m houses a matrix $X \in \mathbb{R}^{m \times n}$; a colour image a $H \times W \times 3$ tensor. The dot product measures alignment between vectors (its normalised version, cosine similarity, is the workhorse of recommendation systems and semantic search), and a matrix–vector product $W\mathbf{x}$ is a *change of representation*: it rotates, stretches, and mixes coordinates. In Module 4 this becomes literal — every neural-network layer *is* one such matrix.

Eigenvalues and dimensionality reduction. Eigenvectors are non-zero directions along which a linear transformation A acts purely by scaling without changing direction:

$$A\mathbf{v} = \lambda \mathbf{v}. \tag{1}$$

The covariance matrix Σ of a dataset is symmetric, so its eigenvectors are mutually orthogonal, and the eigenvector with the largest eigenvalue points along the direction of greatest variance. Projecting the data onto the top eigenvectors is *principal component analysis* (PCA):

$$\mathbf{v}_1 = \arg\max_{\|\mathbf{v}\|=1} \mathbf{v}^\top \Sigma \, \mathbf{v}, \tag{2}$$

i.e. the first principal component is the top eigenvector of Σ. Dropping low-variance directions keeps most of the signal in far fewer coordinates — compression, de-noising, and two-dimensional plots of hundred-dimensional data (Strang 2019, Chapters 5–6). NumPy makes all of it concrete, and its compiled vector operations are orders of magnitude faster than Python loops (Harris et al. 2020).

Hands-on with Python

```python
import numpy as np

X = np.random.default_rng(0).normal(size=(500, 20))
Xc = X - X.mean(axis=0)              # centre the columns
C  = (Xc.T @ Xc) / (X.shape[0]-1)    # covariance matrix
evals, evecs = np.linalg.eigh(C)     # symmetric solver
pcs = evecs[:, ::-1][:, :2]          # top-2 principal components
X2  = Xc @ pcs                       # 20-D data projected to 2-D
print(X2.shape, evals[::-1][:2])     # (500, 2) and top variances
```

2.2 Probability and statistics: the language of uncertainty

Real data is noisy and future observations are stochastic, so predictions in machine learning are fundamentally probabilistic statements.

Expectation, variance, and covariance. For a random variable X, the **expected value** $\mathbb{E}[X] = \int xp(x)\,dx$ (or $\sum x_i P(x_i)$) measures central tendency. The **variance** quantifies dispersion:

$$\mathrm{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2. \tag{3}$$

For two variables X and Y, **covariance** measures joint linear variation $\mathrm{Cov}(X, Y) = \mathbb{E}\big[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\big]$. When normalized by standard deviations $\sigma_X \sigma_Y$, it yields the Pearson correlation coefficient $\rho_{X,Y} \in [-1, 1]$.

Common distributions. A small family covers foundational ML tasks: the *Bernoulli* for binary outcomes $y \in \{0, 1\}$; the *Categorical* for K-class labels and LLM tokens; and the *Gaussian (Normal)* distribution for measurement noise:

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{4}$$

Likelihood, MLE, and MAP estimation. Parameter fitting estimates unknown distribution parameters $\boldsymbol{\theta}$ from data $\mathcal{D} = \{x_1, \ldots, x_m\}$. The **Likelihood function** $L(\boldsymbol{\theta}) = p(\mathcal{D} \mid \boldsymbol{\theta}) = \prod_{i=1}^{m} p(x_i \mid \boldsymbol{\theta})$ evaluates data probability under candidate $\boldsymbol{\theta}$. Maximizing likelihood (or minimizing negative log-likelihood $-\ln L(\boldsymbol{\theta})$) gives the **Maximum Likelihood Estimator (MLE)**:

$$\hat{\boldsymbol{\theta}}_{\mathrm{MLE}} = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \ln p(x_i \mid \boldsymbol{\theta}). \tag{5}$$

When prior knowledge $p(\boldsymbol{\theta})$ is available, **Maximum A Posteriori (MAP)** estimation incorporates the prior via Bayes' theorem:

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} \left[\sum_{i=1}^{m} \ln p(x_i \mid \boldsymbol{\theta}) + \ln p(\boldsymbol{\theta})\right], \tag{6}$$

which directly derives ℓ_2 (Ridge) and ℓ_1 (Lasso) regularisation terms from Gaussian and Laplace priors, respectively.

Bayes' theorem. The foundation of probabilistic updating connects prior beliefs to observed data:

$$P(A \mid B) \;=\; \frac{P(B \mid A)\,P(A)}{P(B)}. \tag{7}$$

Bayes' theorem, in words

Posterior $\propto$ likelihood $\times$ prior: what you believe *after* seeing evidence is proportional to how well each hypothesis would have produced that evidence, weighted by what you believed *before*.

A medical screening example demonstrates why base rates dominate intuition. A diagnostic test with 99% sensitivity and a 5% false-positive rate, applied to a rare disease with 1% prevalence, yields

$$P(\text{disease} \mid +) = \frac{P(+ \mid \text{disease})\,P(\text{disease})}{P(+)} \tag{8}$$

$$= \frac{\overbrace{0.99}^{\text{sensitivity}} \times \overbrace{0.01}^{P(\text{disease})}}{\underbrace{0.99}_{\text{sensitivity}} \times \underbrace{0.01}_{P(\text{disease})} + \underbrace{0.05}_{\text{false pos rate}} \times \underbrace{0.99}_{P(\text{no disease})}} \approx 0.17 : \tag{9}$$

only one positive in six is a true case, because the disease is rare to begin with. Note that the two 0.99 terms in the denominator represent entirely different quantities: test sensitivity $P(+ \mid \text{disease}) = 0.99$ versus healthy prior $P(\text{no disease}) = 1 - 0.01 = 0.99$. The same base-rate logic governs fraud alerts, security anomaly detection, and imbalanced classification.

Hypothesis testing, confidence intervals, and the CLT. To discipline empirical findings:

- **Central Limit Theorem (CLT):** For independent observations with finite variance σ^2, the standard error of the sample mean is exactly $\sigma/\sqrt{n}$. The CLT additionally states that as $n \to \infty$, the standardized sample mean distribution converges to a standard normal distribution $\mathcal{N}(0, 1)$.
- **Confidence Intervals & Bootstrap Resampling:** Rather than relying solely on single-point estimates or p-values (which can be vulnerable to p-hacking under multiple testing), practitioners construct 95% confidence intervals via asymptotic standard errors or non-parametric *bootstrap resampling* (repeatedly sampling data with replacement to estimate metric variance empirical distributions).

Hands-on with Python

```python
import numpy as np

rng = np.random.default_rng(42)
# 10,000 experiments: average of 40 Exponential(1) draws each
means = rng.exponential(scale=1.0, size=(10_000, 40)).mean(axis=1)
print(f"Sample Mean: {means.mean():.3f} | SE: {means.std():.3f}") # ~1.0 & 1/sqrt(40)
```

The standard deviation of sample means matches standard error $\sigma/\sqrt{n}$, illustrating CLT convergence.

2.3 Optimisation: how models learn

A **loss function** $\mathcal{L}(\hat{y}, y)$ scores error on a single instance; the **cost function** $J(\boldsymbol{\theta})$ (empirical risk) averages loss over m samples:

$$J(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i=1}^{m} \big(h_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i\big)^2. \tag{10}$$

The **gradient** $\nabla_{\boldsymbol{\theta}} J$ points in the direction of steepest *ascent*. Parameter updates step in the opposite direction:

$$\boldsymbol{\theta}_{t+1} \;=\; \boldsymbol{\theta}_t \;-\; \eta\, \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t), \tag{11}$$

where $\eta > 0$ is the learning rate. For strictly convex objectives (e.g., OLS linear regression with full-rank X or ℓ_2 regularisation), gradient descent converges to the unique global minimum. For non-convex neural network loss surfaces, stochastic mini-batch gradient descent (SGD) and adaptive optimizers reliably locate high-quality local minima.

Adam and AdamW optimisers. Standard SGD uses a fixed step size. The **Adam** optimiser (Kingma and Ba 2015) maintains exponential moving averages of gradients g_t (first moment m_t) and squared gradients g_t^2 (second moment v_t):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{12}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta\, \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \tag{13}$$

In modern deep learning and Transformer training, **AdamW** (Loshchilov and Hutter 2019) is strongly preferred over standard Adam because it decouples ℓ_2 weight decay regularization from adaptive gradient moment updates, preventing gradient scaling from inadvertently suppressing weight decay penalty terms.

Adam optimiser step from scratch

```python
import numpy as np

def adam_step(theta, grad, m, v, t, lr=0.01,
              beta1=0.9, beta2=0.999, eps=1e-8):
    m = beta1 * m + (1 - beta1) * grad
    v = beta2 * v + (1 - beta2) * (grad ** 2)
    m_hat = m / (1 - beta1 ** t)
    v_hat = v / (1 - beta2 ** t)
    theta_next = theta - (lr * m_hat) / (np.sqrt(v_hat) + eps)
    return theta_next, m, v
```

Adam updates each parameter using bias-corrected first and second moment estimates.

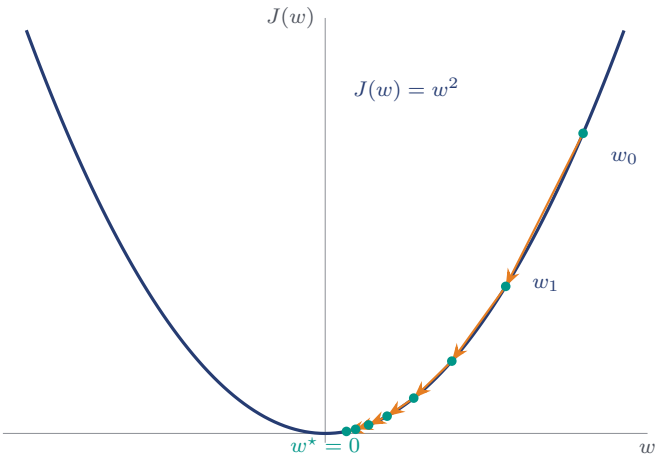


Figure 4. Gradient descent on $J(w) = w^2$ with learning rate $\eta = 0.15$: each step $w \leftarrow w - \eta\, 2w$ shrinks the distance to the minimum by 30%.

Hands-on with Python

```python
w, lr = 5.0, 0.15
for step in range(8):
    grad = 2 * w            # derivative of f(w) = w**2
    w = w - lr * grad       # the update rule, eq. (8)
    print(f"step {step}:  w = {w:.4f}")
    # w -> 0.2882: convergence toward the minimum
```

Check your understanding

1. In plain words, what does an eigenvector of the covariance matrix tell you about a dataset?
2. In the medical-test example, what happens to $P(\text{disease} \mid +)$ as prevalence rises to 10%?
3. What goes wrong, mechanically, when the learning rate in equation (11) is set to 1.1 for $J(w) = w^2$?

Sources & further reading

- G. Strang (2019). *Linear Algebra and Learning from Data*. Wellesley, MA: Wellesley–Cambridge Press
- L. Wasserman (2004). *All of Statistics: A Concise Course in Statistical Inference*. New York: Springer
- S. Boyd and L. Vandenberghe (2004). *Convex Optimization*. Cambridge: Cambridge University Press
- I. Goodfellow et al. (2016). *Deep Learning*. Cambridge, MA: MIT Press
- C. M. Bishop (2006). *Pattern Recognition and Machine Learning*. New York: Springer
- C. R. Harris et al. (2020). “Array Programming with NumPy”. In: *Nature* 585, pp. 357–362
- W. McKinney (2010). “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference (SciPy)*, pp. 56–61
- J. VanderPlas (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol, CA: O’Reilly Media

3 Classical Machine Learning: The Core Engine

Module 3 — at a glance (Estimated Study: 10 hours • Level: Intermediate)

Focus. The algorithms that still run the majority of real-world, structured-data workloads — and the evaluation discipline that separates real learning from memorisation.

Key concepts. Supervised learning: regression, decision trees, SVMs, ensembles. Unsupervised learning: K-Means clustering and anomaly detection. Reinforcement learning: MDPs, multi-armed bandits, exploration vs. exploitation. Evaluation: accuracy, precision, recall, F1, ROC-AUC.

After this module you will be able to choose an appropriate classical algorithm for a task, cluster unlabelled data, solve multi-armed bandit problems, and — most importantly — evaluate any model honestly.

On the tabular, structured data of business (customers, transactions, sensors), classical machine learning frequently still beats glamorous deep models. It is also the best place to internalise the habits — splits, metrics, baselines — that govern everything later (Hastie et al. 2009; Pedregosa et al. 2011).

3.1 Supervised learning

Supervised learning starts from labelled pairs $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$ and a split into *training*, *validation*, and *test* sets. If y is a number the task is *regression*; if y is a category it is *classification*.

Constructing the design matrix X from raw data. Consider a dataset of m observations $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, where each feature vector $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{in})^\top \in \mathbb{R}^n$ represents n input variables. A linear regression model expresses the prediction as an affine combination:

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_n x_{in} = \beta_0 + \boldsymbol{\beta}^\top \mathbf{x}_i, \tag{14}$$

where $\beta_0 \in \mathbb{R}$ is the intercept (bias) and $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_n)^\top \in \mathbb{R}^n$ is the weight vector. To express predictions across all m samples compactly as a single matrix–vector product $\hat{\mathbf{y}} = X\boldsymbol{\beta}_{\text{full}}$, we prepend a dummy constant feature $x_{i0} = 1$ to each sample vector $\tilde{\mathbf{x}}_i = (1, x_{i1}, \ldots, x_{in})^\top \in \mathbb{R}^{n+1}$ and define the parameter vector $\boldsymbol{\beta}_{\text{full}} = (\beta_0, \beta_1, \ldots, \beta_n)^\top \in \mathbb{R}^{n+1}$. Stacking these augmented row vectors $\tilde{\mathbf{x}}_i^\top$ forms the **design matrix** $X \in \mathbb{R}^{m \times (n+1)}$:

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \ldots & x_{1n} \\ 1 & x_{21} & x_{22} & \ldots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \ldots & x_{mn} \end{bmatrix} \in \mathbb{R}^{m \times (n+1)}. \tag{15}$$

Under this representation, the Ordinary Least Squares (OLS) objective minimises the residual sum of squares $\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2$, yielding the closed-form normal equations solution:

$$\hat{\boldsymbol{\beta}}_{\text{OLS}} = (X^\top X)^{-1} X^\top \mathbf{y}, \tag{16}$$

computed via QR factorisation or SVD for numerical stability (Strang 2019).

Feature maps and basis expansion. The matrix X is not restricted to raw measurements. By applying a non-linear feature mapping $\boldsymbol{\phi}(\mathbf{x}) = [\phi_1(\mathbf{x}), \ldots, \phi_d(\mathbf{x})]^\top$, linear models can capture complex non-linear curves. For example, a 1D input x expanded into a degree-d polynomial basis

yields row vectors $\tilde{\mathbf{x}}_i^\top = [1, x_i, x_i^2, \ldots, x_i^d]$, transforming non-linear fitting into a linear problem in feature space $\mathbb{R}^{d+1}$.

Building Design Matrix X & Basis Expansion

```python
import numpy as np

x_raw = np.array([1.5, 2.0, 3.5, 4.0, 5.5])
y = np.array([2.1, 2.9, 6.1, 8.2, 14.5])

# 1. Prepend bias column of 1s -> X in R^(m x 2)
X_linear = np.column_stack([np.ones(len(x_raw)), x_raw])

# 2. Basis expansion: Polynomial features [1, x, x^2, x^3]
X_poly = np.column_stack([x_raw**p for p in range(4)])

# Closed-form OLS via QR decomposition (R beta = Q^T y) & SVD (avoiding inv(X^T X)):
Q, R = np.linalg.qr(X_poly)
beta_poly_qr = np.linalg.solve(R, Q.T @ y)
beta_poly_svd = np.linalg.lstsq(X_poly, y, rcond=None)[0]

print(f"Linear X shape:   {X_linear.shape} | beta: {np.linalg.lstsq(X_linear, y,
    rcond=None)[0].round(2)}")
print(f"Poly X (QR):      {X_poly.shape} | beta: {beta_poly_qr.round(2)}")
print(f"Poly X (SVD):     {X_poly.shape} | beta: {beta_poly_svd.round(2)}")
```

Regularisation: Ridge (ℓ_2) and Lasso (ℓ_1). When features are highly correlated or $n > m$, OLS exhibits high variance. Regularisation fights overfitting by penalising model complexity. Ridge regression adds an ℓ_2 penalty:

$$\hat{\boldsymbol{\beta}}_{\text{Ridge}} = \arg\min_{\boldsymbol{\beta}} \frac{1}{2m}\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \alpha\|\boldsymbol{\beta}\|_2^2, \tag{17}$$

which shrinks all coefficients smoothly toward zero but keeps them non-zero. By contrast, the **Lasso** (ℓ_1 regularisation) penalises the sum of absolute values:

$$\hat{\boldsymbol{\beta}}_{\text{Lasso}} = \arg\min_{\boldsymbol{\beta}} \frac{1}{2m}\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \alpha\|\boldsymbol{\beta}\|_1, \qquad \|\boldsymbol{\beta}\|_1 = \sum_{j=1}^{n} |\beta_j|. \tag{18}$$

Why Lasso induces sparsity. Unlike ℓ_2 sphere contours, the ℓ_1 constraint set $\{\boldsymbol{\beta} : \|\boldsymbol{\beta}\|_1 \leq t\}$ is a geometric polytope (a diamond in 2D) with sharp corners located directly on the coordinate axes. The elliptical level sets of the unconstrained OLS loss expand until they touch the constraint boundary; because the diamond's vertices lie on the axes where one or more coefficients equal zero, the solution frequently lands directly on a corner, driving uninformative feature weights strictly to zero (Hastie et al. 2009). Analytically, the absolute value function $|\beta_j|$ is non-differentiable at zero, yielding a subdifferential interval $[-1, 1]$. Coordinate descent solves for β_j via the *soft-thresholding operator*:

$$\mathcal{S}_\gamma(z) = \operatorname{sign}(z)\max\big(0,\ |z| - \gamma\big), \tag{19}$$

which clamps small feature correlations below $\gamma = \alpha$ to exactly zero, performing automated feature selection.

Lasso (ℓ_1) Regularisation & Numerical Solvers

```python
import numpy as np

# Synthetic dataset with collinearity & sparse ground truth
rng = np.random.default_rng(42)
m, n = 100, 5
X = rng.normal(size=(m, n))
X[:, 4] = X[:, 0] + X[:, 1] + 1e-4 * rng.normal(size=m) # collinear feature
true_w = np.array([3.0, -2.5, 1.5, 0.0, 0.0])
y = X @ true_w + 0.2 * rng.normal(size=m)

# 1. Numerically stable OLS Normal Equations via QR & SVD
Q, R = np.linalg.qr(X)
beta_qr = np.linalg.solve(R, Q.T @ y)             # QR decomposition solver
beta_svd = np.linalg.lstsq(X, y, rcond=None)[0]   # SVD solver

# 2. Lasso via Coordinate Descent with Soft-Thresholding
def soft_threshold(z, gamma):
    return np.sign(z) * np.maximum(0, np.abs(z) - gamma)

def lasso_cd(X, y, alpha, max_iter=500):
    m, n = X.shape
    w = np.zeros(n)
    norm_x2 = np.sum(X**2, axis=0)
    for _ in range(max_iter):
        for j in range(n):
            r_j = y - (X @ w - X[:, j] * w[j])     # partial residual
            rho_j = X[:, j] @ r_j
            w[j] = soft_threshold(rho_j, m * alpha) / norm_x2[j]
    return w

w_lasso_np = lasso_cd(X, y, alpha=0.1)

print("True weights: ", true_w)
print("OLS QR:       ", beta_qr.round(2))          # unstable on collinear feature
print("NumPy Lasso:  ", w_lasso_np.round(2))       # zeroes noise & collinear weights
```

Classification: Sigmoid, Logistic Regression, and Softmax. When target labels y are categorical, linear output scores $z = \boldsymbol{\beta}^\top \mathbf{x} + \beta_0$ are unbounded and unsuitable for probabilities.

Binary Logistic Regression. For two classes $y \in \{0, 1\}$, the **sigmoid (logistic) function** maps any real score $z \in \mathbb{R}$ into a valid probability $p \in (0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{e^z + 1}. \tag{20}$$

The model interprets $p = P(y = 1 \mid \mathbf{x})$, implying that linear log-odds are given by $\ln\left(\frac{p}{1-p}\right) = \boldsymbol{\beta}^\top \mathbf{x} + \beta_0$. Fitting parameters via Maximum Likelihood Estimation yields the **Binary Cross-Entropy (BCE)** cost function:

$$\mathcal{L}(\boldsymbol{\beta}) = -\frac{1}{m} \sum_{i=1}^{m} \left[y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i) \right], \qquad \hat{y}_i = \sigma(\boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0). \tag{21}$$

Multinomial Logistic Regression and Softmax. For $K > 2$ discrete classes $y \in \{1, \ldots, K\}$, the model outputs a vector of unnormalised scores (logits) $\mathbf{z} = W\mathbf{x} + \mathbf{b} \in \mathbb{R}^K$, where $W \in \mathbb{R}^{K \times n}$. The **Softmax function** exponentiates and normalises logits into a categorical probability distribution

$\hat{\mathbf{p}} = [\hat{p}_1, \ldots, \hat{p}_K]^\top$:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K. \tag{22}$$

Key properties of Softmax:

1. **Positivity and Normalisation:** Exponentiation enforces $e^{z_k} > 0$, and dividing by $\sum_j e^{z_j}$ guarantees $\sum_{k=1}^{K} \hat{p}_k = 1$ on the probability simplex.
2. **Temperature Scaling:** Incorporating a temperature parameter $T > 0$ as $\text{softmax}(\mathbf{z}/T)$ controls confidence: as $T \to 0^+$ predictions sharpen toward a one-hot argmax, whereas high $T \to \infty$ produces a uniform distribution (a mechanism central to LLM sampling in Module 4).
3. **Generalisation of Sigmoid:** For $K = 2$, Softmax on $[z_1, z_2]$ simplifies identically to Sigmoid on $(z_1 - z_2)$.

Training multinomial logistic regression minimises the **Categorical Cross-Entropy** loss:

$$\mathcal{L}(W, \mathbf{b}) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_{i,k} \ln \hat{p}_{i,k}, \tag{23}$$

where $y_{i,k} \in \{0, 1\}$ is the one-hot target indicator for class k.

Sigmoid, Softmax & Cross-Entropy Loss from Scratch

```python
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def softmax(Z, temp=1.0):
    # Subtract max logit per row for numerical stability
    Z_scaled = Z / temp
    exp_Z = np.exp(Z_scaled - np.max(Z_scaled, axis=-1, keepdims=True))
    return exp_Z / np.sum(exp_Z, axis=-1, keepdims=True)

# 3 samples, 4 classes (logits Z in R^(3 x 4))
logits = np.array([[2.0, 1.0, 0.1, 3.0],
                   [0.5, 4.0, 1.2, 0.0],
                   [-1.0, 0.0, 5.0, 0.2]])

probs = softmax(logits, temp=1.0)
y_true = np.array([3, 1, 2]) # True targets for each sample

# Categorical Cross-Entropy Loss: -log(p_true)
nll = -np.log(probs[np.arange(len(y_true)), y_true])
print("Probabilities:\n", probs.round(3))
print(f"Per-sample loss: {nll.round(3)} | Mean: {nll.mean():.3f}")
```

Naive Bayes and k-NN baselines. Bridging directly from Bayes' theorem (equation 7), the **Naive Bayes classifier** computes the posterior probability of class $y \in \mathcal{Y}$ given features $\mathbf{x} = (x_1, \ldots, x_n)$ by assuming features are conditionally independent given y:

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} \; P(y \mid \mathbf{x}) = \arg\max_{y \in \mathcal{Y}} \; P(y) \prod_{j=1}^{n} P(x_j \mid y). \tag{24}$$

Depending on data types, feature likelihoods $P(x_j \mid y)$ assume specific parametric distributions:

- **Gaussian Naive Bayes** for continuous features: $P(x_j \mid y) = \frac{1}{\sqrt{2\pi\sigma_{y,j}^2}} \exp\left(-\frac{(x_j-\mu_{y,j})^2}{2\sigma_{y,j}^2}\right)$, where $\mu_{y,j}$ and $\sigma_{y,j}^2$ are class-conditional empirical means and variances.
- **Multinomial & Bernoulli Naive Bayes** for discrete token counts or binary indicators in NLP.

To prevent zero-frequency probability collapse on unseen features during inference, **Laplace smoothing** adds a prior pseudo-count $\alpha \geq 1$: $\hat{P}(x_j \mid y) = \frac{N_{y,x_j}+\alpha}{N_y+\alpha \cdot d}$. In practice, products of small probabilities cause numerical underflow; computations sum log-likelihoods instead: $\ln P(y) + \sum_{j=1}^{n} \ln P(x_j \mid y)$.

The **k-Nearest Neighbors (k-NN)** algorithm is a non-parametric instance-based baseline that stores all training observations $\mathcal{D}$. For a query point $\mathbf{x}$, it identifies the set $\mathcal{N}_k(\mathbf{x})$ of k closest samples under a distance metric (e.g., Euclidean distance $d_2(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_2$ or Manhattan distance $d_1(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_1$) and assigns the majority class vote:

$$\hat{y} = \operatorname{mode}\big(\{y_i : \mathbf{x}_i \in \mathcal{N}_k(\mathbf{x})\}\big). \tag{25}$$

Hyperparameter k controls the bias–variance trade-off: $k = 1$ creates complex, low-bias decision boundaries prone to overfitting noise, whereas large k over-smooths class margins (high bias). While k-NN requires zero training compute, inference cost scales linearly as $O(m \cdot n)$ per query; spatial indexing trees (KD-Trees, Ball Trees) accelerate spatial queries in low-to-moderate dimensions ($n \ll 50$).

Gaussian Naive Bayes from Scratch & k-NN Baselines

```python
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

class GaussianNBScratch:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.priors = np.array([np.mean(y == c) for c in self.classes])
        self.means = np.array([X[y == c].mean(axis=0) for c in self.classes])
        self.vars = np.array([X[y == c].var(axis=0) + 1e-9 for c in self.classes])
        return self

    def predict(self, X):
        # Log likelihood for numerical stability: log P(y) + sum log P(x_j|y)
        log_probs = []
        for i, c in enumerate(self.classes):
            log_prior = np.log(self.priors[i])
            log_pdf = -0.5 * np.sum(np.log(2 * np.pi * self.vars[i]) +
                                    ((X - self.means[i])**2) / self.vars[i], axis=1)
            log_probs.append(log_prior + log_pdf)
        return self.classes[np.argmax(np.array(log_probs), axis=0)]

# Synthetic binary dataset
rng = np.random.default_rng(42)
X = rng.normal(size=(200, 4))
y = (X[:, 0] + X[:, 1] > 0).astype(int)

gnb = GaussianNBScratch().fit(X[:150], y[:150])
knn = KNeighborsClassifier(n_neighbors=5).fit(X[:150], y[:150])

print(f"Gaussian NB (Scratch) Acc: {(gnb.predict(X[150:]) == y[150:]).mean():.3f}")
print(f"k-NN (k=5) Test Acc:       {knn.score(X[150:], y[150:]):.3f}")
```

Naive Bayes computes class posterior probabilities in log-space via Gaussian feature likelihoods, matching k-NN baseline performance on smooth distributions.

Decision trees and ensembles. A decision tree partitions feature space recursively into axis-aligned rectangular regions using a sequence of greedy binary splits. At node m, the algorithm searches over all features j and split thresholds t to maximize impurity reduction $\Delta I = I(m) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$, where $|S|$ is node sample count. Common classification impurity metrics for class proportions $\mathbf{p} = (p_1, \ldots, p_K)$ include:

$$\text{Gini Impurity:} \quad I_G(\mathbf{p}) = 1 - \sum_{k=1}^{K} p_k^2, \qquad \text{Entropy:} \quad H(\mathbf{p}) = -\sum_{k=1}^{K} p_k \log_2 p_k. \tag{26}$$

For regression trees, splits minimize variance (MSE) $I_{\text{MSE}}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2$. Unconstrained trees grow until leaves are pure, causing severe overfitting (high variance). Overfitting is controlled via pre-pruning constraints (`max_depth`, `min_samples_split`) or cost-complexity post-pruning $R_\alpha(T) = R(T) + \alpha |T|$.

To overcome single-tree overfitting, **ensemble methods** combine multiple weak tree learners (Hastie et al. 2009):

- **Random Forests (Bagging):** Grows B independent de-correlated trees on bootstrap samples of training rows, evaluating only a random subset of $\sqrt{n}$ features per split (Breiman 2001). Averaging tree outputs reduces variance by factor $\text{Var}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, where ρ is pairwise tree correlation.
- **Gradient Boosting (XGBoost / LightGBM):** Builds trees sequentially: $\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(\mathbf{x}_i)$, where step t fits tree f_t to pseudo-residuals $r_{i,t} = -\left[\frac{\partial L(y_i, \hat{y})}{\partial \hat{y}}\right]_{\hat{y}^{(t-1)}}$ (Chen and Guestrin 2016). Gradient boosting achieves state-of-the-art accuracy on tabular data.

Decision Trees, Gini Impurity & Ensembles

```python
import numpy as np
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier

def gini_impurity(y):
    """Compute Gini Impurity I_G(p) = 1 - sum(p_k^2)"""
    if len(y) == 0: return 0.0
    p = np.bincount(y) / len(y)
    return 1.0 - np.sum(p**2)

# Synthetic classification dataset with non-linear interaction
rng = np.random.default_rng(42)
X = rng.normal(size=(500, 10))
y = (X[:, 0]*2 - X[:, 1] + X[:, 2]**2 > 1.0).astype(int)

rf = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42).fit(X[:400],
    y[:400])
gb = HistGradientBoostingClassifier(max_iter=100, max_depth=5,
    random_state=42).fit(X[:400], y[:400])

print(f"Root Split Gini Impurity:   {gini_impurity(y[:400]):.3f}")
print(f"Random Forest Test Acc:     {rf.score(X[400:], y[400:]):.3f}")
print(f"Gradient Boosting Test Acc: {gb.score(X[400:], y[400:]):.3f}")
```

Ensembles aggregate decision tree splits; gradient boosting iteratively corrects prior tree errors to model complex non-linear feature interactions.

Support Vector Machines (SVMs). The SVM finds the decision boundary $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = 0$ maximizing the margin $\frac{2}{\|\mathbf{w}\|_2}$ to the nearest training points (Cortes and Vapnik 1995). For non-

linearly separable data, the **soft-margin formulation** introduces slack variables $\xi_i \geq 0$:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{m}\xi_i \qquad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \tag{27}$$

where target labels $y_i \in \{-1, +1\}$ and hyperparameter $C > 0$ trades off margin maximization against training errors. Mathematically, soft-margin SVM minimizes unconstrained **Hinge Loss** plus ℓ_2 regularization: $\mathcal{L} = \sum_{i=1}^{m} \max\big(0, 1 - y_i f(\mathbf{x}_i)\big) + \frac{1}{2C}\|\mathbf{w}\|_2^2$.

Formulating the dual problem using Lagrange multipliers $\alpha_i \geq 0$ yields:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m}\alpha_i - \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha_i\alpha_j y_i y_j \big(\mathbf{x}_i^\top \mathbf{x}_j\big) \qquad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{m}\alpha_i y_i = 0. \tag{28}$$

Observations with $\alpha_i > 0$ form the **support vectors** lying on or inside the margin. The dual formulation reveals that predictions depend exclusively on dot products $\mathbf{x}_i^\top \mathbf{x}_j$. Replacing this dot product with a non-linear kernel function $K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j)$ — the **Kernel Trick** — allows SVMs to construct non-linear decision boundaries in high-dimensional Hilbert spaces without explicitly computing expensive feature maps $\boldsymbol{\phi}(\cdot)$:

- **Linear Kernel:** $K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$.
- **Polynomial Kernel:** $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^d$.
- **Radial Basis Function (RBF / Gaussian Kernel):** $K(\mathbf{x}, \mathbf{z}) = \exp\big(-\gamma\|\mathbf{x} - \mathbf{z}\|_2^2\big)$, where $\gamma > 0$ controls the inverse length scale (higher γ yields a smaller effective radius and tighter decision boundaries; lower γ yields broader spatial influence).

Support Vector Machines: Linear vs. RBF Kernel

```python
import numpy as np
from sklearn.svm import SVC

# Generate non-linearly separable concentric circles dataset
rng = np.random.default_rng(42)
X = rng.uniform(-2, 2, size=(300, 2))
y = ((X[:, 0]**2 + X[:, 1]**2) < 1.5).astype(int)

# Compare Linear vs RBF Kernel SVM
svm_lin = SVC(kernel='linear', C=1.0).fit(X[:240], y[:240])
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale').fit(X[:240], y[:240])

print(f"Linear Kernel Test Acc: {svm_lin.score(X[240:], y[240:]):.3f}")
print(f"RBF Kernel Test Acc:    {svm_rbf.score(X[240:], y[240:]):.3f}")
print(f"RBF Support Vectors:    {len(svm_rbf.support_vectors_)} / 240 samples")
```

The RBF kernel projects non-linearly separable points into implicit infinite-dimensional space, creating complex circular decision boundaries governed by support vectors.

Feature scaling requirement. Algorithms that calculate spatial distances (K-Means, k-NN, SVM) or compute gradient steps (Logistic/Linear Regression, Neural Networks) are highly sensitive to feature scales: features with large magnitudes (e.g., house area in sq ft) dominate Euclidean metrics over features with small ranges (e.g., room count). Scaling is generally critical for distance-based algorithms (k-NN, SVM, K-Means) and often beneficial for gradient-based optimization (Logistic/Linear Regression, Neural Networks), while tree-based models are largely scale-invariant.

Table 4. Choosing a first algorithm for tabular data.

Algorithm	Reach for it when	Strengths	Watch out
Linear / logistic regression	Baselines; small, clean data	Fast, interpretable, statistically principled	Underfits complex nonlinear structure
Naive Bayes	Text classification, fast baseline	Very fast, handles high dimensions	Strong feature independence assumption
k-NN	Small datasets, distance baselines	No training phase; flexible boundaries	Slow inference $O(mn)$; scale-sensitive
Decision tree	Interpretability is the product	Human-readable rules; no feature scaling	Single trees overfit badly
Random forest	Default robust choice	Variance reduction via bagging; few knobs	Less interpretable; larger memory
Gradient boosting (XGBoost)	Maximum accuracy on tables	State of the art on structured data	Needs tuning; can overfit small data
SVM	Small–medium data, many features	Max-margin generalisation; kernels	Scales poorly to very large datasets

⚖ CRITICAL: Avoid Data Leakage

Fit scalers on training data ONLY. A ubiquitous beginner error is fitting a scaler on the entire dataset prior to splitting. This causes *data leakage*: information from the test set (mean μ and standard deviation σ) leaks into the training process, yielding artificially optimistic evaluation metrics. Always compute scaling parameters using `fit_transform()` on the training split, and apply `transform()` on validation/test splits. Use `sklearn.pipeline.Pipeline` to automate this safely.

3.2 Unsupervised learning

Unsupervised learning discovers underlying structure without target labels.

K-Means & Silhouette Score. The canonical partitioning algorithm **K-Means** (MacQueen 1967) minimizes within-cluster inertia $J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$ via Expectation-Maximization step updates. Cluster separation quality is quantified by the **Silhouette Score** $s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$.

Density-based & Mixture Models: DBSCAN & GMMs. K-Means assumes spherical, equal-sized clusters. To identify arbitrary non-convex clusters, **DBSCAN** (Ester et al. 1996) groups points based on density (ϵ-neighborhoods and minimum core samples), naturally isolating noise outliers. For probabilistic soft-clustering, **Gaussian Mixture Models (GMMs)** represent data distributions as a linear combination of K Gaussians $\sum \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \Sigma_k)$, fitted via the Expectation-Maximization (EM) algorithm (Bishop 2006).

Non-linear Dimensionality Reduction: t-SNE & UMAP. While PCA computes linear orthogonal projections, non-linear manifolds require non-linear embedding algorithms:

- **t-SNE** (Maaten and Hinton 2008) converts pairwise Euclidean distances into conditional probabilities under Student-t distributions, preserving local neighborhood structures in 2D/3D visualizations.
- **UMAP** (McInnes et al. 2018) constructs fuzzy simplicial sets to optimize low-dimensional layouts, preserving both local and global structure with superior scalability.

Anomaly Detection. Methods include Isolation Forest (Liu et al. 2008) (random feature splits

isolating anomalies near root nodes), Elliptic Envelope, and Autoencoders (Module 4).

DBSCAN, K-Means & Silhouette Score

```python
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score
import numpy as np

# Fit K-Means and DBSCAN on synthetic data
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X_tr)
dbscan = DBSCAN(eps=0.5, min_samples=5).fit(X_tr)

score = silhouette_score(X_tr, kmeans.labels_)
print(f"K-Means Silhouette Score: {score:.3f}")
print(f"DBSCAN Clusters Found:   {len(set(dbscan.labels_)) - (1 if -1 in dbscan.labels_
    else 0)}")
```

3.3 Reinforcement learning

While supervised learning requires target pairs $(\mathbf{x}_i, y_i)$ and unsupervised learning discovers unlabelled structure, **Reinforcement Learning (RL)** optimizes sequential decision-making through trial-and-error environment interaction (Sutton and Barto 2018). An agent observes state $s_t \in \mathcal{S}$, executes action $a_t \in \mathcal{A}$ under policy $\pi(a \mid s)$, transitions to s_{t+1}, and receives reward r_{t+1}.

MDPs & Bellman Equations. Environments obey the Markov property: $P(S_{t+1} = s', R_{t+1} = r \mid S_t, A_t, \dots) = P(S_{t+1} = s', R_{t+1} = r \mid S_t, A_t)$. An MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ defines state/action spaces, transition dynamics $\mathcal{P}(s' \mid s, a)$, reward function $\mathcal{R}$, and discount factor $\gamma \in [0, 1)$. Value functions $V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ and $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$ satisfy the **Bellman Optimality Equation**:

$$Q^\star(s, a) = \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma \max_{a' \in \mathcal{A}} Q^\star(s', a') \right]. \tag{29}$$

Temporal Difference (TD) Learning, SARSA, & Q-Learning. Model-free RL learns directly from interaction:

- **Monte Carlo vs. TD Learning:** Monte Carlo updates values after full episode completion, whereas TD learning updates predictions incrementally from immediate reward plus estimated next-state value (bootstrapping): $V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$.
- **SARSA (On-Policy) vs. Q-Learning (Off-Policy):** SARSA updates Q-values using the actual next action executed by the policy: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$. Off-policy Q-learning (Watkins and Dayan 1992) updates using the maximum possible next-state action value $\max_a Q(s_{t+1}, a)$.

Actor-Critic Methods & Advantage Functions. To reduce policy gradient variance, **Actor-Critic** architectures maintain an *Actor* (policy $\pi_{\boldsymbol{\theta}}(a \mid s)$) and a *Critic* (value function $V_{\boldsymbol{\phi}}(s)$). The update scales gradients by the **Advantage function**:

$$A(s, a) = Q(s, a) - V(s) \;\approx\; r_{t+1} + \gamma V(s_{t+1}) - V(s_t), \tag{30}$$

quantifying how much better action a is compared to the average expected value of state s. Modern algorithms like Proximal Policy Optimization (PPO) (Schulman et al. 2017) clip policy updates to prevent destabilizing gradient steps, serving as the core engine for RLHF (Module 4).

Clarification on AlphaGo & MCTS. Note that Monte Carlo Tree Search (MCTS) in AlphaGo (Silver et al. 2016) performs lookahead search over a perfect game simulator, combining heuristic rollouts with neural evaluation. This lookahead search over a simulator is distinct from model-based RL

algorithms that must explicitly learn an empirical transition probability model $\hat{\mathcal{P}}(s' \mid s, a)$ from raw environment data.

3.4 Modern evaluation methodology & diagnostics

Evaluating a model requires moving beyond simple train/test accuracy.

Classification Metrics: Confusion Matrix, PR-AUC, and ROC-AUC. Precision $\frac{TP}{TP+FP}$, Recall $\frac{TP}{TP+FN}$, $F_1 = 2\frac{P \cdot R}{P+R}$, and ROC-AUC measure classification performance. For severely imbalanced datasets (e.g., fraud, rare diseases), ROC-AUC can present an overly optimistic score because false positives are divided by a vast number of true negatives (TN). The **Precision-Recall Curve (PR-AUC)** is strongly preferred for imbalanced evaluation because it focuses exclusively on the positive class.

Probabilistic Calibration & Proper Scoring Rules. Model probabilities $\hat{p} \in [0, 1]$ should match real-world frequencies: among events assigned $\hat{p} = 0.8$, exactly 80% should occur. **Calibration curves** (reliability diagrams) plot predicted probability bins against empirical frequencies. Quantitative calibration error is measured by the **Brier Score** $\frac{1}{m}\sum(\hat{y}_i - y_i)^2$ and Categorical Cross-Entropy (Log-Loss), both of which are *proper scoring rules* minimized strictly when predicted probabilities equal true posterior distributions.

Advanced Resampling & Data Drift. Standard k-fold cross-validation fails on structured data dependencies:

- **Nested Cross-Validation:** Outer folds evaluate generalization error while inner folds perform hyperparameter optimization, preventing optimistic hyperparameter selection bias.
- **Time-Series & Grouped CV:** Time-dependent data requires rolling-window splits ($t_{\text{train}} < t_{\text{test}}$) to prevent temporal leakage; grouped data (multiple samples per user) requires `GroupKFold` to prevent user leakage across splits.
- **Data Drift & Concept Drift:** In production, input feature distributions change over time (*covariate shift* $P(X)$ changes) or target relationships alter (*concept drift* $P(Y \mid X)$ changes), requiring monitoring statistics (Kolmogorov-Smirnov tests, Population Stability Index) and scheduled model retraining.

Check your understanding

1. For a cancer-screening tool, which metric would you prioritise, and what is the cost of that choice?
2. Why does K-Means struggle with two concentric rings of points, and what does that reveal about its assumptions?
3. In a K-armed bandit problem, how does increasing ϵ in an ϵ-greedy strategy impact short-term vs. long-term return?
4. Your cross-validation scores vary from 0.62 to 0.91 across folds. What does that variance tell you, and what would you do next?

Sources & further reading

- T. M. Mitchell (1997). *Machine Learning*. New York: McGraw-Hill
- T. Hastie et al. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer
- C. M. Bishop (2006). *Pattern Recognition and Machine Learning*. New York: Springer
- C. Cortes and V. Vapnik (1995). "Support-Vector Networks". In: *Machine Learning* 20.3, pp. 273–297
- L. Breiman (2001). "Random Forests". In: *Machine Learning* 45.1, pp. 5–32

- T. Chen and C. Guestrin (2016). “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794
- J. MacQueen (1967). “Some Methods for Classification and Analysis of Multivariate Observations”. In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1, pp. 281–297
- F. T. Liu et al. (2008). “Isolation Forest”. In: *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pp. 413–422
- R. S. Sutton and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press
- C. J. C. H. Watkins and P. Dayan (1992). “Q-Learning”. In: *Machine Learning* 8.3-4, pp. 279–292
- V. Mnih et al. (2015). “Human-level Control Through Deep Reinforcement Learning”. In: *Nature* 518.7540, pp. 529–533
- J. Schulman et al. (2017). “Proximal Policy Optimization Algorithms”. In
- D. Silver et al. (2016). “Mastering the Game of Go with Deep Neural Networks and Tree Search”. In: *Nature* 529.7587, pp. 484–489
- D. Amodei et al. (2016). “Concrete Problems in AI Safety”. In: *arXiv preprint arXiv:1606.06565*
- T. Fawcett (2006). “An Introduction to ROC Analysis”. In: *Pattern Recognition Letters* 27.8, pp. 861–874
- F. Pedregosa et al. (2011). “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12, pp. 2825–2830

4 Transition to Deep Learning and Modern AI

Module 4 — at a glance (Estimated Study: 14 hours • Level: Advanced)

Focus. The leap from engineered features to learned representations and self-supervised pretraining: neural networks, autoencoders, contrastive learning, transformer architectures, and the mechanics of generative foundation models.

Key concepts. Perceptrons, backpropagation; representation learning, manifold hypothesis, autoencoders, latent spaces, Word2Vec; self-supervised learning, InfoNCE loss, pretext tasks; CNNs, RNNs, transformers; scaling laws, LoRA, and LLM inference.

After this module you will be able to explain how neural networks discover continuous latent representations, derive self-supervised pretext tasks and contrastive losses, explain why transformers and SSL together unlocked large language models, and trace LLM generation step-by-step.

The jump from classical ML to deep learning is a jump in *who designs the features*. A classical pipeline is given hand-crafted inputs; a deep network learns a hierarchy of them — edges, then textures, then parts, then objects (LeCun et al. 2015). What engineering achieved by hand, depth achieves with data and compute.

4.1 Neural networks: perceptrons, activations, backpropagation

The **perceptron** (Rosenblatt 1958) is a single neuron: a weighted sum passed through a threshold,

$$\hat{y} = \mathbb{1}\{\mathbf{w}^\top \mathbf{x} + b > 0\} \in \{0, 1\} \quad (\text{or } \operatorname{sgn}(\mathbf{w}^\top \mathbf{x} + b) \in \{-1, +1\}). \tag{31}$$

It can learn any function that a straight line can separate — and provably cannot learn XOR, a limitation that chilled the field's first winter (Minsky and Papert 1969). The repair is to *stack* units and interleave **nonlinear activation functions** between layers:

$$\mathbf{a}^{(\ell)} = \phi\left(W^{(\ell)}\, \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \qquad \ell = 1, \dots, L. \tag{32}$$

Without ϕ, the stack would collapse into a single linear map. The rectified linear unit, $\phi(z) = \max(0, z)$, became the default for being cheap, non-saturating, and sparse in its activations (Nair and Hinton 2010). Such networks are universal approximators (Hornik et al. 1989) — a statement about *capacity*, with no guarantee that training finds good parameters.

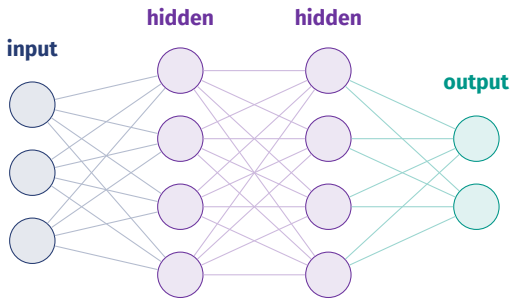


Figure 5. A fully connected 3–4–4–2 network. Every edge carries a learnable weight; each layer applies its matrix, then a nonlinearity.

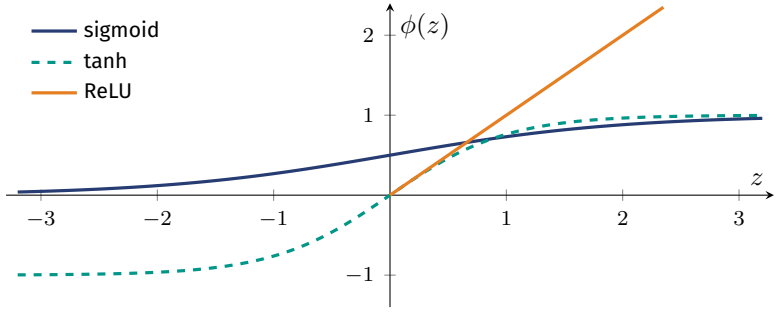


Figure 6. Common activation functions. ReLU's lack of saturation for $z > 0$ helped make deep networks trainable at scale (Nair and Hinton 2010).

Backpropagation (Rumelhart et al. 1986) computes the gradient of the loss $\mathcal{L}$ with respect to every weight (the chain rule from Module 1) organised backwards through the layers. With $\delta^{(\ell)} \equiv \partial \mathcal{L} / \partial \mathbf{z}^{(\ell)}$:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot \phi'\big(\mathbf{z}^{(L)}\big), \tag{33}$$

$$\boldsymbol{\delta}^{(\ell)} = \big(W^{(\ell+1)\top} \boldsymbol{\delta}^{(\ell+1)}\big) \odot \phi'\big(\mathbf{z}^{(\ell)}\big), \qquad \ell = L-1, \dots, 1, \tag{34}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \, \mathbf{a}^{(\ell-1)\top}, \tag{35}$$

where $\odot$ is element-wise multiplication. Backprop is a gradient-*computation* algorithm; the learning itself is still gradient descent (equation (11)), now run on mini-batches through frameworks such as PyTorch or JAX.

Vanishing and exploding gradients. In deep multi-layer architectures, backpropagating loss gradients across L layers involves repeated matrix products backwards:

$$\boldsymbol{\delta}^{(1)} = \left(\prod_{\ell=2}^{L} W^{(\ell)\top} \operatorname{diag}\Big(\phi'\Big(\mathbf{z}^{(\ell)}\Big)\Big) \right) \boldsymbol{\delta}^{(L)}. \tag{36}$$

Backpropagated signals scale with matrix singular values and activation derivatives ϕ'. In naive unregularized deep stacks, if effective singular values or activation slopes remain consistently below 1, gradients decay exponentially to zero ($\boldsymbol{\delta}^{(1)} \to \mathbf{0}$, *vanishing gradients*), leaving early layers untrained. Conversely, if effective products exceed 1, gradients explode exponentially ($\boldsymbol{\delta}^{(1)} \to \infty$, *exploding gradients*). In modern deep networks, gradient dynamics depend strongly on architecture, initialization, normalization, and activation distributions. Deep networks employ explicit safeguards — residual skip connections (He et al. 2016), gradient clipping, Layer Normalization, Xavier/He weight initialization (Glorot and Bengio 2010; He et al. 1915), and non-saturating activations (ReLU) (Nair and Hinton 2010) — to maintain stable gradient norm propagation.

Two-Layer MLP with Manual Backpropagation on Non-Linear Data

```python
import numpy as np

# Non-linear XOR / concentric circle dataset
rng = np.random.default_rng(42)
X_b = rng.uniform(-2, 2, size=(200, 2))
y_b = ((X_b[:, 0]**2 + X_b[:, 1]**2) < 1.2).astype(int).reshape(-1, 1)

# Initialize weights with He Normalization (scaling by sqrt(2/fan_in))
W1 = rng.normal(0, np.sqrt(2/2), size=(2, 8)); b1 = np.zeros((1, 8))
W2 = rng.normal(0, np.sqrt(2/8), size=(8, 1)); b2 = np.zeros((1, 1))

for epoch in range(500):
    # Forward pass: Input -> Hidden (ReLU) -> Output (Sigmoid)
    z1 = X_b @ W1 + b1; a1 = np.maximum(0, z1)   # ReLU activation
    z2 = a1 @ W2 + b2;  hat_y = 1 / (1 + np.exp(-np.clip(z2, -500, 500)))

    # Backward pass (Gradient computation via chain rule)
    dz2 = (hat_y - y_b) / len(X_b)               # dL/dz2 for BCE loss
    dW2 = a1.T @ dz2; db2 = dz2.sum(axis=0, keepdims=True)
    da1 = dz2 @ W2.T;  dz1 = da1 * (z1 > 0)      # ReLU derivative
    dW1 = X_b.T @ dz1; db1 = dz1.sum(axis=0, keepdims=True)

    # Gradient descent parameter updates
    W1 -= 0.1 * dW1; b1 -= 0.1 * db1
    W2 -= 0.1 * dW2; b2 -= 0.1 * db2

bce_loss = -np.mean(y_b*np.log(hat_y+1e-7) + (1-y_b)*np.log(1-hat_y+1e-7))
acc = np.mean((hat_y > 0.5) == y_b)
print(f"Epoch 500 | BCE Loss: {bce_loss:.4f} | Accuracy: {acc:.3f}")
```

4.2 Representation learning: from manual engineering to latent spaces

In classical machine learning (Module 3), algorithm accuracy hinges on manual feature engineering — crafting input matrix columns $\mathbf{x} \in \mathbb{R}^n$ using domain heuristics (e.g., SIFT descriptors in vision, TF-IDF in NLP, polynomial features). Manual feature engineering is laborious, task-specific, fragile, and fails to scale to high-dimensional unstructured inputs like pixels or continuous audio waveforms. **Representation learning** (Bengio et al. 2013) eliminates manual feature engineering: neural networks discover parameterized continuous transformations $\mathbf{z} = f_{\boldsymbol{\theta}}(\mathbf{x}) \in \mathbb{R}^d$ that automatically extract latent semantic structures and data-generating invariants.

The Manifold Hypothesis: linear vs. non-linear projections. High-dimensional real-world data vectors $\mathbf{x} \in \mathbb{R}^n$ (such as $256 \times 256 \times 3$ RGB images in $\mathbb{R}^{196,608}$) occupy an infinitesimal fraction of ambient space. The **Manifold Hypothesis** posits that real-world data concentrates near a smooth, low-dimensional non-linear manifold $\mathcal{M} \subset \mathbb{R}^n$ of intrinsic dimension $d \ll n$. Linear dimensionality reduction (PCA, Module 2) computes orthogonal projections $\mathbf{z} = W^\top \mathbf{x}$, which capture global variance but fail to unfold non-linear manifolds (e.g., Swiss Roll geometry). Interleaving non-linear activation functions across hidden layers allows deep networks to learn non-linear coordinate charts, flattening complex curved manifolds in input space into linearly separable representations in latent space $\mathcal{Z}$.

The Core Principle of Representation Learning

Instead of hand-crafting task-specific features, representation learning trains neural network backbones to transform raw inputs $\mathbf{x}$ into continuous vector representations $\mathbf{z} \in \mathbb{R}^d$. Good representations disentangle underlying variation factors, compress noise, and make downstream tasks (classification, search, generation) trivial to solve using linear layers or distance metrics.

Autoencoders and Latent Space Geometry. An **Autoencoder** is a neural network trained to reconstruct its input through an architectural bottleneck:

$$\text{Encoder:} \quad \mathbf{z} = g_{\boldsymbol{\phi}}(\mathbf{x}) \in \mathbb{R}^d, \qquad \text{Decoder:} \quad \hat{\mathbf{x}} = f_{\boldsymbol{\theta}}(\mathbf{z}) \in \mathbb{R}^n. \tag{37}$$

Minimizing reconstruction loss $\mathcal{L}_{\text{rec}}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$ (or Binary Cross-Entropy) forces the bottleneck $\mathbf{z}$ to discard uninformative noise while preserving dominant structural variations:

- **Undercomplete Autoencoders:** Enforcing bottleneck dimension $d < n$ prevents identity memorization. For linear encoders/decoders under MSE loss, the learned bottleneck subspace is equivalent to PCA.
- **Denoising Autoencoders (DAEs):** To prevent memorization when $d \geq n$, DAEs (Vincent et al. 2008) corrupt inputs $\tilde{\mathbf{x}} \sim q(\tilde{\mathbf{x}} \mid \mathbf{x})$ via noise injection or random masking, training the decoder to reconstruct clean $\mathbf{x}$. DAEs learn a score-matching vector field $\nabla_{\mathbf{x}} \ln p(\mathbf{x})$ that projects off-manifold points back onto data manifold $\mathcal{M}$.
- **Variational Autoencoders (VAEs):** A probabilistic formulation (Kingma and Welling 2014) where the encoder outputs parameters $\boldsymbol{\mu}(\mathbf{x})$ and $\boldsymbol{\sigma}^2(\mathbf{x})$ of a latent Gaussian distribution $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2))$. Training maximizes the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}_{\text{ELBO}}(\boldsymbol{\phi}, \boldsymbol{\theta}; \mathbf{x}) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\big[\ln p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})\big] \; - \; D_{\text{KL}}(q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \,||\, p(\mathbf{z}))\,, \tag{38}$$

  where $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I)$ and D_{KL} penalizes latent distributions that deviate from standard normal priors. Reparameterization $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ ($\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$) enables gradient backpropagation through stochastic sampling.

Distributed Representations & Word Embeddings. Symbolic NLP represents words as orthogonal one-hot vectors $\mathbf{e}_i \in \{0,1\}^{|V|}$, where dot products $\mathbf{e}_i^\top \mathbf{e}_j = 0$ for all $i \neq j$, failing to capture semantic similarity. **Distributed representations** (Mikolov et al. 2013) embed discrete tokens into dense continuous spaces $\mathbb{R}^d$ ($d \ll |V|$), mapping semantic relationships directly to vector angles and distances.

The **Word2Vec Skip-Gram** architecture predicts context tokens w_{t+j} given target token w_t within window size c. To avoid computing full-vocabulary softmax denominators over $|V|$, **Negative Sampling (SGNS)** reformulates training as binary logistic regressions distinguishing true context tokens from K noise words sampled from unigram distribution $P_n(w)$:

$$\mathcal{L}_{\text{SGNS}} = -\sum_{t=1}^{T} \sum_{-c \leq j \leq c, j \neq 0} \left[\ln \sigma\left(\mathbf{v}'^{\top}_{w_{t+j}} \mathbf{v}_{w_t} \right) + \sum_{k=1}^{K} \mathbb{E}_{w_n \sim P_n(w)} \left[\ln \sigma\left(-\mathbf{v}'^{\top}_{w_n} \mathbf{v}_{w_t} \right) \right] \right], \tag{39}$$

where $\mathbf{v}_w$ and $\mathbf{v}'_w$ denote target and context vectors. An emergent property of SGNS spaces is **linear vector compositionality**: semantic analogies correspond to parallel displacement vectors ($\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$).

Metric Learning & Triplet Loss. Representation learning can also optimize explicit distance metrics in embedding space. **Triplet Loss** trains encoders on anchor $\mathbf{a}$, positive $\mathbf{p}$ (same class), and negative $\mathbf{n}$ (different class) samples, enforcing that positive pairs remain closer than negative pairs by margin $\alpha > 0$:

$$\mathcal{L}_{\text{triplet}} = \max\Big(0, \; \|\mathbf{z}_a - \mathbf{z}_p\|_2^2 - \|\mathbf{z}_a - \mathbf{z}_n\|_2^2 + \alpha\Big). \tag{40}$$

Deep Autoencoder Feature Extraction & Latent Space Analysis

```python
import numpy as np
from sklearn.linear_model import LogisticRegression

# 1. Generate non-linear 10D synthetic dataset with 2D intrinsic manifold
rng = np.random.default_rng(42)
t = rng.uniform(-np.pi, np.pi, size=600)
# Non-linear 2D manifold embedded in 10D ambient space
X_raw = np.column_stack([np.sin(t), np.cos(t), t**2, np.sin(2*t), np.cos(2*t),
                         t, np.exp(t/5), np.sin(3*t), np.cos(3*t), t**3/10])
y_labels = (np.sin(t) + np.cos(2*t) > 0).astype(int)

# 2. Define 10D -> 4D -> 2D -> 4D -> 10D Deep Autoencoder in NumPy
W_e1 = rng.normal(0, np.sqrt(2/10), size=(10, 4)); b_e1 = np.zeros((1, 4))
W_e2 = rng.normal(0, np.sqrt(2/4), size=(4, 2));   b_e2 = np.zeros((1, 2))  # Bottleneck z
    in R^2
W_d1 = rng.normal(0, np.sqrt(2/2), size=(2, 4));   b_d1 = np.zeros((1, 4))
W_d2 = rng.normal(0, np.sqrt(2/4), size=(4, 10));  b_d2 = np.zeros((1, 10))

# 3. Train Autoencoder via SGD with MSE Reconstruction Loss
lr = 0.05
for epoch in range(400):
    # Forward Pass
    h1 = np.maximum(0, X_raw @ W_e1 + b_e1)           # Encoder L1 (ReLU)
    z  = h1 @ W_e2 + b_e2                             # Bottleneck representation z in R^2
    h2 = np.maximum(0, z @ W_d1 + b_d1)               # Decoder L1 (ReLU)
    X_hat = h2 @ W_d2 + b_d2                          # Reconstructed output in R^10

    # Backward Pass (Chain rule gradients)
    dX_hat = 2 * (X_hat - X_raw) / len(X_raw)
    dW_d2 = h2.T @ dX_hat;         db_d2 = dX_hat.sum(axis=0, keepdims=True)
    dh2   = (dX_hat @ W_d2.T) * (h2 > 0)
    dW_d1 = z.T @ dh2;             db_d1 = dh2.sum(axis=0, keepdims=True)
    dz    = dh2 @ W_d1.T
    dW_e2 = h1.T @ dz;             db_e2 = dz.sum(axis=0, keepdims=True)
    dh1   = (dz @ W_e2.T) * (h1 > 0)
    dW_e1 = X_raw.T @ dh1;         db_e1 = dh1.sum(axis=0, keepdims=True)

    # Parameter Updates
    W_e1 -= lr * dW_e1; b_e1 -= lr * db_e1; W_e2 -= lr * dW_e2; b_e2 -= lr * db_e2
    W_d1 -= lr * dW_d1; b_d1 -= lr * db_d1; W_d2 -= lr * dW_d2; b_d2 -= lr * db_d2

# 4. Extract 2D latent representation z and evaluate linear separability
z_latent = np.maximum(0, X_raw[:450] @ W_e1 + b_e1) @ W_e2 + b_e2
z_test   = np.maximum(0, X_raw[450:] @ W_e1 + b_e1) @ W_e2 + b_e2

clf_raw = LogisticRegression().fit(X_raw[:450], y_labels[:450])
clf_z   = LogisticRegression().fit(z_latent, y_labels[:450])

print(f"Epoch 400 Reconstruction MSE: {np.mean((X_hat - X_raw)**2):.4f}")
print(f"Linear Classifier Acc (Raw 10D Features):   {clf_raw.score(X_raw[450:],
    y_labels[450:]):.3f}")
print(f"Linear Classifier Acc (Learned 2D Latent z): {clf_z.score(z_test,
    y_labels[450:]):.3f}")
```

4.3 Architectures: CNNs, RNNs, and transformers

Convolutional networks (CNNs) exploit spatial structure in grid data (images, signals) via local receptive fields and weight sharing. A convolution slides a small learnable filter K over input I:

$$(I * K)(i,j) = \sum_m \sum_n I(i-m,\, j-n)\, K(m,n). \tag{41}$$

Sliding the filter confers built-in **translation equivariance** ($(I*K)(x-\Delta) = ((I \text{ shifted}) * K)(x)$): shifting an input shifts its feature map identically. Spatial pooling and hierarchical aggregation then build approximate **translation invariance** at higher layers. LeNet established document recognition (LeCun et al. 1998); AlexNet's GPU depth ignited modern computer vision (Krizhevsky et al. 2012).

Recurrent networks (RNNs) handle sequences by carrying a hidden state:

$$\mathbf{h}_t = \tanh\!\big(W_h\, \mathbf{h}_{t-1} + W_x\, \mathbf{x}_t + \mathbf{b}\big). \tag{42}$$

Training across long sequences suffers from vanishing gradients; the **LSTM**'s gated memory cells provided the decisive fix (Hochreiter and Schmidhuber 1997), dominating sequential modeling for two decades.

Transformers replaced sequential recurrent state updates with parallel *self-attention*, positional encodings, and feed-forward transformations (Vaswani et al. 2017). Each token emits a query Q, a key K, and a value V; the output for each position is a weighted sum of all values, with weights given by query–key compatibility:

$$\text{Attention}(Q,K,V) = \text{softmax}\!\left(\frac{Q\,K^\top}{\sqrt{d_k}}\right) V. \tag{43}$$

Because every token attends to all others in a single parallel step, transformers train vastly faster than RNNs on modern parallel hardware. However, computing pairwise attention scores across sequence length n incurs $O(n^2)$ computational and memory complexity, creating a bottleneck for long *context windows*.

Scaled Dot-Product Attention from Scratch in NumPy

```python
import numpy as np

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = (Q @ K.T) / np.sqrt(d_k)
    if mask is not None:
        scores = np.where(mask == 0, -1e9, scores)
    weights = np.exp(scores - np.max(scores, axis=-1, keepdims=True))
    weights /= weights.sum(axis=-1, keepdims=True)  # Softmax
    return weights @ V, weights

# Demonstrate with 4 tokens of dimension 8
seq_len, d_k = 4, 8
Q = K = V = np.random.default_rng(0).standard_normal((seq_len, d_k))
causal_mask = np.tril(np.ones((seq_len, seq_len)))  # Lower-triangular mask
out, attn = scaled_dot_product_attention(Q, K, V, mask=causal_mask)
print("Attention Output Shape:", out.shape)
print("Causal Attention Weights Matrix:\n", np.round(attn, 3))
```

Positional encodings. Plain self-attention is permutation-invariant. To inject sequence order, **positional encodings** are added to input embeddings. The original Transformer used sinusoidal

frequencies (Vaswani et al. 2017). Modern LLMs (e.g., Llama 3) predominantly use **Rotary Position Embeddings (RoPE)** (Su et al. 2024), rotating query and key vectors in complex 2D planes to encode relative distance directly into $QK^\top$.

Residual connections, normalization, and optimization. Deep Transformer stacks survive vanishing gradients by wrapping every sub-layer with a **residual (skip) connection** (He et al. 2016) and **Layer Normalization**. While original Transformers used **Post-Layer Normalization** ($\mathbf{y} = \mathrm{LayerNorm}(\mathbf{x} + \mathcal{F}(\mathbf{x}))$), modern foundation models standardly use **Pre-Layer Normalization** (Pre-LN):

$$\mathbf{y} = \mathbf{x} + \mathcal{F}\big(\mathrm{LayerNorm}(\mathbf{x})\big). \tag{44}$$

Pre-LN preserves an unattenuated identity gradient highway ($\mathbf{x}$ passed directly to addition), enabling stable training across hundreds of layers without delicate warm-up schedules. Model optimization relies on **AdamW** (Loshchilov and Hutter 2019) with learning rate warmup and cosine decay schedules.

4.4 Self-supervised learning: unifying supervised and unsupervised paradigms

The transition from classical machine learning to modern foundation models rests fundamentally on **Self-Supervised Learning (SSL)**. Classical supervised learning achieves high accuracy but is strictly bottlenecked by expensive, non-scalable human annotations $(\mathbf{x}_i, y_i)$. Traditional unsupervised learning (K-Means, PCA) scales to unlabelled data but struggles to discover task-relevant high-level semantics without targets. Self-supervised learning bridges this divide by turning unlabelled raw data into its own supervisor: the model automatically constructs artificial target signals (*pretext tasks*) directly from intrinsic structural patterns within raw inputs $\mathbf{x}$.

Self-Supervised Learning: The Foundation Model Engine

Self-Supervised Learning converts unlabelled raw data (text, images, audio) into supervised learning targets by hiding, masking, or augmenting parts of the input and training a network to predict the missing signal. SSL provides an unlimited training signal, turning web-scale raw datasets into universal pre-trained representations.

Taxonomy of SSL Pretext Objectives. SSL transforms raw signals into target supervision through three primary mathematical formulations:

1. **Causal (Autoregressive) Language Modeling (CLM):** The model predicts token x_t conditioned on preceding historical context $\mathbf{x}_{1:t-1}$, minimizing negative log-likelihood over sequence length T:

$$\mathcal{L}_{\mathrm{CLM}}(\boldsymbol{\theta}) = -\sum_{t=1}^{T} \ln P_{\boldsymbol{\theta}}(x_t \mid x_1, x_2, \ldots, x_{t-1}). \tag{45}$$

   Pioneered at scale by the GPT family (Radford et al. 2018; Brown et al. 2020), CLM creates generative autoregressive models capable of next-token prediction and zero/few-shot in-context learning.

2. **Masked Reconstruction (MLM & MAE):** A subset of positions $\mathcal{M}$ (typically 15% in text, 75% in vision) is corrupted or masked out. The network predicts original values x_i using bidirectional context $\mathbf{x}_{\setminus\mathcal{M}}$:

$$\mathcal{L}_{\mathrm{MLM}}(\boldsymbol{\theta}) = -\sum_{i \in \mathcal{M}} \ln P_{\boldsymbol{\theta}}(x_i \mid \mathbf{x}_{\setminus\mathcal{M}}). \tag{46}$$

   Pioneered by BERT (Devlin et al. 2019) in NLP and Masked Autoencoders (MAE) (He et al. 2022) in computer vision, masked reconstruction forces bidirectional attention layers to model deep contextual dependencies.

3. **Contrastive Learning & InfoNCE Loss:** Beyond text, SSL for vision and multimodal data generates two stochastic augmented views $(\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}'_i)$ of the same image (positive pair) via cropping, color jittering, and blurring. Encoders project views into normalized latent vectors $\mathbf{z}_i = g(f(\tilde{\mathbf{x}}_i))$. The **InfoNCE Loss** (Oord et al. 2018; Chen et al. 2020) pulls positive pairs together while pushing negative views $\mathbf{z}_k^-$ (other images in the batch) apart:

$$\mathcal{L}_{\text{InfoNCE}} = -\sum_{i=1}^{N} \ln \frac{\exp\left(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau\right)}{\exp\left(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau\right) + \sum_{k \neq i} \exp\left(\text{sim}(\mathbf{z}_i, \mathbf{z}_k^-)/\tau\right)}, \tag{47}$$

where $\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$ is cosine similarity and temperature hyperparameter $\tau > 0$ controls the penalty hardness assigned to difficult negative samples.

Non-contrastive SSL & Collapse Prevention. Contrastive learning requires large batch sizes (e.g., 4096 in SimCLR (Chen et al. 2020)) to supply sufficient negative samples. Non-contrastive methods (BYOL, SimSiam) eliminate negative pairs entirely, training an online network to predict target network representations. Representation collapse (where the encoder outputs a constant vector $\mathbf{z}_i = \mathbf{c}$) is prevented using asymmetrical predictor heads, moving-average target networks, and *stop-gradient* operations ($\text{sg}[\cdot]$).

The Symbiotic Relationship between SSL and Transformers. The simultaneous rise of Transformers (Vaswani et al. 2017) and SSL represents a profound technological synergy. SSL provided an infinite supply of training supervision from web text, while the Transformer provided the exact architectural mechanisms required to ingest that scale:

- **Elimination of Recurrent Bottlenecks:** RNNs process input sequences sequentially ($\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$), requiring $O(N)$ sequential operations during training. Backpropagation Through Time (BPTT) creates severe memory bottlenecks. Transformers compute all pairwise self-attention interactions in parallel ($O(1)$ sequential depth), unlocking full GPU throughput across trillions of SSL tokens.
- **Unrestricted Inductive Bias & Scaling Laws:** CNNs enforce local spatial kernels, and RNNs enforce strict temporal decay. While helpful on small datasets, rigid inductive biases cap model capacity on massive datasets. Transformers replace rigid priors with dynamic soft self-attention. When combined with SSL, Transformer capacity scales predictably according to empirical power laws without early performance saturation (Kaplan et al. 2020; Hoffmann et al. 2022).
- **The Pretrain–Finetune Paradigm Shift:** SSL converts raw unlabelled data into universal cognitive representations. The modern AI paradigm uses SSL to pretrain massive base Transformer models, which transfer efficiently to downstream tasks via Supervised Fine-Tuning (SFT), Parameter-Efficient Adaptation (LoRA (Hu et al. 2022)), or zero/few-shot in-context learning (Brown et al. 2020).

SimCLR Contrastive Learning with InfoNCE Loss from Scratch

```python
import numpy as np

# 1. Define vectorized InfoNCE Loss with Temperature Scaling (SimCLR style)
def infonce_loss(z1, z2, temperature=0.1):
    """
    z1, z2: (N, d) L2-normalized embedding projections for 2 augmented views
    """
    N = len(z1)
    # Concatenate both views -> 2N x d matrix
    z = np.vstack([z1, z2])

    # Compute 2N x 2N cosine similarity matrix (since vectors are L2-normalized)
    sim_matrix = (z @ z.T) / temperature

    # Mask out self-similarity along main diagonal
    np.fill_diagonal(sim_matrix, -1e9)

    # Positive pair target indices: row i pairs with (i + N) % (2N)
    labels = np.arange(2 * N)
    labels[:N] += N
    labels[N:] -= N

    # Numerically stable Log-Softmax Cross-Entropy Loss
    max_sim = np.max(sim_matrix, axis=1, keepdims=True)
    exp_sim = np.exp(sim_matrix - max_sim)
    log_prob = sim_matrix - max_sim - np.log(np.sum(exp_sim, axis=1, keepdims=True))

    loss = -np.mean(log_prob[np.arange(2 * N), labels])
    return loss

# 2. Simulate 2 augmented views of 4 samples in 8-dimensional projection space
rng = np.random.default_rng(42)
N, d = 4, 8
# Base representations
z_base = rng.normal(size=(N, d))
z_base /= np.linalg.norm(z_base, axis=1, keepdims=True)

# Positive views (small random augmentation perturbation)
z1 = z_base + 0.05 * rng.normal(size=(N, d))
z2 = z_base + 0.05 * rng.normal(size=(N, d))

# L2 normalize projection vectors
z1 /= np.linalg.norm(z1, axis=1, keepdims=True)
z2 /= np.linalg.norm(z2, axis=1, keepdims=True)

loss_aligned = infonce_loss(z1, z2, temperature=0.1)

# Unaligned random representations baseline
z_rand = rng.normal(size=(N, d))
z_rand /= np.linalg.norm(z_rand, axis=1, keepdims=True)
loss_unaligned = infonce_loss(z1, z_rand, temperature=0.1)

print(f"InfoNCE Loss (Aligned Positive Pairs):    {loss_aligned:.4f}")
print(f"InfoNCE Loss (Random Unaligned Pairs):   {loss_unaligned:.4f}")
```

4.5 Generative AI & Foundation Models: Adaptation and Inference

The Foundation Model Paradigm Shift. Building upon self-supervised pretraining, foundation models map raw data into dense representations that adapt across diverse downstream tasks:

$$\text{Supervised ML} \longrightarrow \text{Self-Supervised Pretraining} \longrightarrow \text{Supervised Fine-Tuning (SFT)} \longrightarrow \text{Preference Optimizatio} \tag{48}$$

Embeddings & Contextual Semantics. Static word embeddings (word2vec (Mikolov et al. 2013)) showed linear vector arithmetic ($w_{\text{king}} - w_{\text{man}} + w_{\text{woman}} \approx w_{\text{queen}}$). Modern contextual embeddings in Transformers dynamically compute representations dependent on surrounding sequence context.

Latent Space & Diffusion Models. Generative models compress data into lower-dimensional latent representations. Variational Autoencoders (Kingma and Welling 2014) and GANs (Goodfellow et al. 2014) pioneered latent generation. **Diffusion Models** (Ho et al. 2020) dominate image/audio synthesis by training U-Nets or Vision Transformers to reverse a forward Gaussian noise process $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$.

Large Language Models & Causal Masking. Autoregressive LLMs predict the next token via cross-entropy loss over vocabulary V:

$$p(y = j \mid \mathbf{x}) = \frac{\exp(z_j)}{\sum_{k=1}^{V} \exp(z_k)}, \qquad \mathcal{L}_{\text{CE}} = -\log p(y_{\text{true}} \mid \mathbf{x}). \tag{49}$$

Causal attention masking ($M_{ij} = -\infty$ for $j > i$) prevents tokens from viewing future positions.

Pretraining, Scaling, and Fine-Tuning Taxonomy. Pretraining consumes trillions of tokens. Compute-optimal **Chinchilla scaling analysis** (Hoffmann et al. 2022) demonstrated that model capacity and training token count should scale in roughly equal $1:1$ proportion under compute-optimal budgets. Following pretraining, alignment transforms raw base models into helpful assistants:

1. **Supervised Fine-Tuning (SFT):** Fine-tuning on curated demonstration pairs.
2. **Preference Optimization (RLHF & DPO):** Reinforcement Learning from Human Feedback (Christiano et al. 2017; Ouyang et al. 2022) or Direct Preference Optimization (DPO) (Rafailov et al. 2023) optimizes policy preference loss directly without fitting an explicit reward model.
3. **Parameter-Efficient Fine-Tuning (PEFT / LoRA): LoRA** (Hu et al. 2022) freezes pretrained weight matrices $W_0 \in \mathbb{R}^{d \times k}$ and injects trainable low-rank decomposition matrices $\Delta W = BA$ ($B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$), reducing trainable parameters by $> 99\%$. **QLoRA** (Dettmers et al. 2024) quantizes base model weights to 4-bit NormalFloat while preserving full-precision low-rank adapters.

Modern LLM Inference & Generation Control. At inference, token sampling controls decoding:

- **Temperature Scaling & Greedy Decoding:** Softmax $\text{softmax}(\mathbf{z}/T)$ controls distribution entropy. As $T \to 0^+$, the distribution converges to a one-hot vector at $\arg\max \mathbf{z}$, which code libraries alias as `temperature=0` (greedy decoding).
- **Sampling Strategies:** Top-k truncates sampling to the k most probable tokens; Top-p (nucleus) samples from the smallest candidate set comprising cumulative probability p.
- **Inference Acceleration:** High-throughput inference relies on **KV Caching**[1], continuous batching, quantization (INT8/INT4), and speculative decoding (using a small draft model to generate candidate tokens verified in parallel by a larger target model).
- **Retrieval-Augmented Generation (RAG):** RAG (Lewis et al. 2020) grounds generation by retrieving relevant document snippets from vector databases into prompt context. However,

[1] KV Caching stores past K and V tensors in memory, making incremental attention computation per generated token linear in context length $O(N)$ instead of $O(N^2)$.

RAG reduces but does not eliminate hallucinations: retrieval can fail, retrieved documents can contain errors, or the generator can misinterpret context.

How an LLM writes one token

1. Tokenise the prompt into sub-word tokens.
2. Embed each token into a vector (plus RoPE positional encoding).
3. Pass the sequence through stacked Pre-LN transformer blocks (causal self-attention + FFN).
4. Map final position hidden state through linear head and softmax over vocabulary V.
5. Sample next token via Temperature / Top-p or select argmax greedy token ($T \to 0^+$).
6. Append token to sequence and update KV-cache for the next step.

Big models, big responsibilities: Hallucinations & Contamination

Language models can generate fluent but factually false claims (**hallucinations**). Distinguish factuality errors, faithfulness errors, and retrieval failures. Audit evaluation datasets for benchmark contamination.

Check your understanding

1. Remove the nonlinearity from an MLP: what functions can the resulting network represent, and why?
2. In an undercomplete Autoencoder with linear layers and MSE loss, what classical linear algebra technique does the bottleneck space replicate?
3. Explain how Skip-Gram Negative Sampling (SGNS) overcomes the computational bottleneck of full-vocabulary softmax in Word2Vec.
4. Give two concrete reasons transformers train faster than RNNs on parallel hardware.
5. Mathematically, what role does temperature τ play in the InfoNCE loss for contrastive learning?
6. Why does Self-Supervised Learning (SSL) depend so critically on the parallel self-attention mechanism of Transformers compared to sequential RNNs?

Sources & further reading

- F. Rosenblatt (1958). “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain”. In: *Psychological Review* 65.6, pp. 386–408
- D. E. Rumelhart et al. (1986). “Learning Representations by Back-Propagating Errors”. In: *Nature* 323.6088, pp. 533–536
- K. Hornik et al. (1989). “Multilayer Feedforward Networks Are Universal Approximators”. In: *Neural Networks* 2.5, pp. 359–366
- V. Nair and G. E. Hinton (2010). “Rectified Linear Units Improve Restricted Boltzmann Machines”. In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pp. 807–814
- Y. Bengio et al. (2013). “Representation Learning: A Review and New Perspectives”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8, pp. 1798–1828
- P. Vincent et al. (2008). “Extracting and Composing Robust Features with Denoising Autoencoders”. In: *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pp. 1096–1103
- Y. LeCun et al. (1998). “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11, pp. 2278–2324
- S. Hochreiter and J. Schmidhuber (1997). “Long Short-Term Memory”. In: *Neural Computation* 9.8,

pp. 1735–1780
- K. He et al. (2016). “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778
- A. Vaswani et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems 30*, pp. 5998–6008
- A. van den Oord et al. (2018). “Representation Learning with Contrastive Predictive Coding”. In: *arXiv preprint arXiv:1807.03748*
- T. Chen et al. (2020). “A Simple Framework for Contrastive Learning of Visual Representations”. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 1597–1607
- J. Devlin et al. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 4171–4186
- A. Radford et al. (2018). “Improving Language Understanding by Generative Pre-Training”. In: *OpenAI Blog*
- A. Dosovitskiy et al. (2021). “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations (ICLR)*
- K. He et al. (2022). “Masked Autoencoders Are Scalable Vision Learners”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009
- J. Su et al. (2024). “RoFormer: Enhanced Transformer with Rotary Position Embedding”. In: *Neurocomputing* 568, p. 127063
- T. Mikolov et al. (2013). “Distributed Representations of Words and Phrases and Their Compositionality”. In: *Advances in Neural Information Processing Systems 26*, pp. 3111–3119
- D. P. Kingma and M. Welling (2014). “Auto-Encoding Variational Bayes”. In: *International Conference on Learning Representations (ICLR)*, pp. 1–9
- I. Goodfellow et al. (2014). “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems 27*, pp. 2672–2680
- J. Ho et al. (2020). “Denoising Diffusion Probabilistic Models”. In: *Advances in Neural Information Processing Systems 33*, pp. 6840–6851
- T. B. Brown et al. (2020). “Language Models Are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems 33*, pp. 1877–1901
- J. Hoffmann et al. (2022). “Training Compute-Optimal Large Language Models”. In: *arXiv preprint arXiv:2203.15556*
- L. Ouyang et al. (2022). “Training Language Models to Follow Instructions with Human Feedback”. In: *Advances in Neural Information Processing Systems 35*, pp. 27730–27744
- R. Rafailov et al. (2023). “Direct Preference Optimization: Your Language Model is Secretly a Reward Model”. In: *Advances in Neural Information Processing Systems 36*
- E. J. Hu et al. (2022). “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations (ICLR)*
- T. Dettmers et al. (2024). “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *Advances in Neural Information Processing Systems 36*
- P. Lewis et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33*, pp. 9459–9474
- X. Glorot and Y. Bengio (2010). “Understanding the Difficulty of Training Deep Feedforward Neural Networks”. In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256
- I. Loshchilov and F. Hutter (2019). “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations (ICLR)*

Where to go next

The guide ends; the field does not. A few well-marked paths, matched to what you now know:

- **Strengthen the mathematics:** (Strang 2019; Wasserman 2004; Boyd and Vandenberghe 2004).
- **Go deep on classical ML:** (Hastie et al. 2009; Bishop 2006; Pedregosa et al. 2011).
- **Deep learning proper:** (Goodfellow et al. 2016; Bishop and Bishop 2024; Zhang et al. 2023).
- **Build with LLMs, LoRA & RAG:** Hugging Face ecosystem (Wolf et al. 2020), LoRA (Hu et al. 2022), and RAG (Lewis et al. 2020), read alongside (Vaswani et al. 2017; Brown et al. 2020; Ouyang et al. 2022; Rafailov et al. 2023).
- **Stay grounded:** (Christian 2020; Bender et al. 2021).

Practical Coding Projects Series

To accompany the theoretical foundation developed across the four modules, a complete series of six substantial hands-on coding projects is provided in the repository's `projects/` directory (https://github.com/DanielBerns/AI-course). Each project bridges theoretical concepts to runnable Python implementation using industry-standard frameworks (`NumPy`, `Pandas`, `Scikit-Learn`, `XGBoost`, `PyTorch`, and `Transformers`), with explicit emphasis on data cleaning, exploratory visualization, model diagnostics, resumable training, and inference pipelines.

Project 1: Data Pipeline, Exploratory Analysis & Algorithmic Bias Audit

Curriculum Mapping: Module 1 (The AI Landscape, Compute/Cloud, Data Pipelines, Ethics & Safety).

Overview & Learning Objectives: Hands-on data engineering and ethical auditing pipeline preceding model training on clinical dataset.

Project 2: Math Engine from Scratch (Linear Algebra, Stats & Optimization)

Curriculum Mapping: Module 2 (Mathematics and Data Foundations). Pure NumPy matrix calculus, PCA, Adam/AdamW optimizers.

Project 3: Classical Machine Learning Pipeline & Diagnostics

Curriculum Mapping: Module 3 (Classical Machine Learning). Supervised classification, DBSCAN/K-Means clustering, IsoForest anomaly detection, learning curves.

Project 4: Deep Learning Foundations & Resumable Training Pipeline

Curriculum Mapping: Module 4 (Neural Networks, Backpropagation, PyTorch/Keras). Manual matrix backprop engine, PyTorch MLP, checkpoint manager.

Project 5: Computer Vision & Sequential Architectures (CNNs & LSTMs)

Curriculum Mapping: Module 4 (CNNs and RNNs/LSTMs). PyTorch 2D CNN with Pre-LN Residual Skip Connections ($\mathbf{y} = \mathbf{x} + \mathcal{F}(\text{LayerNorm}(\mathbf{x}))$) and PyTorch LSTM.

Project 6: Transformer Architectures & LLM Fine-Tuning Pipeline

Curriculum Mapping: Module 4 (Transformers, Generative AI & LLMs). Causal self-attention from scratch, RoPE, Hugging Face fine-tuning (SFT/LoRA), next-token sampling engine.

Appendix: Answer Key for Check Your Understanding

This appendix provides concise model answers for all self-check exercises across the four modules.

Module 1: Introduction and the AI Landscape

1. **Classification:** Chess engine, spam filter, and autonomous reasoning demos are all Narrow AI.
2. **GPU vs CPU throughput:** GPUs possess thousands of concurrent lightweight arithmetic cores optimized for parallel matrix tensor operations ($A\mathbf{x} + \mathbf{b}$).
3. **Job screening harms & mitigations:** (1) Historical bias propagation $\rightarrow$ audit for Equalized Odds / proxy features; (2) Representation gaps $\rightarrow$ human oversight & threshold calibration.

Module 2: Mathematics and Data Foundations

1. **Eigenvector of Covariance Matrix:** Specifies principal direction of maximum variance projection without linear distortion.
2. **Bayes' Theorem with 10% Prevalence:** $P(\text{disease} \mid +) = \frac{0.99 \times 0.10}{(0.99 \times 0.10) + (0.05 \times 0.90)} \approx 0.6875$ (68.8%).
3. **Divergence with $\eta = 1.1$ for $J(w) = w^2$:** Step $w_{t+1} = -1.2 w_t$ flips sign and expands magnitude by $1.2\times$, diverging to $\pm\infty$.

Module 3: Classical Machine Learning

1. **Cancer Screening Metric:** Prioritize Recall to minimize False Negatives; precision drops (more false alarms requiring follow-up tests).
2. **K-Means on Concentric Rings:** K-Means assumes convex spherical clusters; concentric rings require density-based clustering (DBSCAN) or spectral methods.
3. **K-Armed Bandit ϵ Trade-off:** Higher ϵ accelerates early exploration but caps asymptotic long-term return due to persistent suboptimal random actions.
4. **Cross-Validation Score Variance (0.62 to 0.91):** Indicates model instability or fold sample imbalance; use stratified k-fold, stronger regularization, or more data.

Module 4: Transition to Deep Learning and Modern AI

1. **MLP without Nonlinearity:** Collapses mathematically into a single linear map $\mathbf{a}^{(L)} = W_{\text{effective}}\mathbf{x}$, restricted to linear hyperplanes.
2. **Linear Undercomplete Autoencoder Subspace:** Under MSE loss, a linear undercomplete autoencoder bottleneck learns the exact principal subspace as Principal Component Analysis (PCA).
3. **Skip-Gram Negative Sampling (SGNS):** Converts expensive full-vocabulary softmax over $|V|$ into $K + 1$ binary logistic regressions per step, replacing $O(|V|)$ denominator summations with $O(K)$ binary updates.
4. **Transformers vs RNNs Training Speed:** (1) Full sequence parallelization via matrix multiplication ($QK^\top$); (2) $O(1)$ direct attention paths between arbitrary positions.
5. **InfoNCE Temperature τ:** Scales cosine similarities before log-softmax, controlling the relative gradient penalty weight assigned to hard negative samples vs. easy negatives.

6. **SSL & Transformer Synergy:** SSL relies on pseudo-tasks (CLM/MLM) over unlabelled web-scale corpora containing trillions of tokens. Sequential RNNs process inputs sequentially ($O(N)$ steps) with memory bottlenecks in BPTT, making web-scale training computationally intractable. Transformers compute all pairwise token interactions in parallel ($O(1)$ sequential operations) and lack rigid local/temporal inductive biases, enabling parameter capacity to absorb massive scale without early performance saturation.

Glossary

Table 5. Quick reference for the vocabulary used in this guide.

Term	Meaning
Artificial intelligence (AI)	Broad field: systems that perform tasks associated with intelligent behaviour.
Machine learning (ML)	Sub-field: algorithms whose performance on a task improves from data.
Deep learning (DL)	ML with multi-layer neural networks that learn hierarchical representations.
Feature / label	Input measurement / target answer supplied during supervised training.
Model	Parameterised function fitted to data.
Loss function	Scalar score of single-example error; cost function averages loss over data.
Gradient	Vector of partial derivatives indicating the direction of steepest ascent.
Backpropagation	Efficient chain-rule algorithm computing loss gradients for all weights layer by layer.
Overfitting	Memorising training data; high training performance, poor generalization.
Embedding	Dense vector representation encoding semantic relationships in continuous space.
Token	Sub-word unit that language models process and generate.
Parameter	Learnable weight/bias inside a model; LLMs contain billions of parameters.
Fine-tuning	Adapting a pretrained model to a specific task via SFT, LoRA, or DPO.
Hallucination	Model output that is fluent and syntactically convincing but factually false.

References

Amodei, D., C. Olah, J. Steinhardt, et al. (2016). "Concrete Problems in AI Safety". In: *arXiv preprint arXiv:1606.06565*.

Bender, E. M., T. Gebru, A. McMillan-Major, and S. Shmitchell (2021). "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pp. 610–623.

Bengio, Y., A. Courville, and P. Vincent (2013). "Representation Learning: A Review and New Perspectives". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8, pp. 1798–1828.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. New York: Springer.

Bishop, C. M. and H. Bishop (2024). *Deep Learning: Foundations and Concepts*. Cham: Springer.

Boyd, S. and L. Vandenberghe (2004). *Convex Optimization*. Cambridge: Cambridge University Press.

Breiman, L. (2001). "Random Forests". In: *Machine Learning* 45.1, pp. 5–32.

Brown, T. B., B. Mann, N. Ryder, et al. (2020). "Language Models Are Few-Shot Learners". In: *Advances in Neural Information Processing Systems 33*, pp. 1877–1901.

Buolamwini, J. and T. Gebru (2018). "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification". In: *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*, pp. 77–91.

Chen, T. and C. Guestrin (2016). "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.

Chen, T., S. Kornblith, M. Norouzi, and G. Hinton (2020). "A Simple Framework for Contrastive Learning of Visual Representations". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 1597–1607.

Christian, B. (2020). *The Alignment Problem: Machine Learning and Human Values*. New York: W. W. Norton & Company.

Christiano, P. F., J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei (2017). "Deep Reinforcement Learning from Human Preferences". In: *Advances in Neural Information Processing Systems 30*, pp. 4299–4307.

Cortes, C. and V. Vapnik (1995). "Support-Vector Networks". In: *Machine Learning* 20.3, pp. 273–297.

Dettmers, T., A. Pagnoni, A. Holtzman, and L. Zettlemoyer (2024). "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems 36*.

Devlin, J., M.-W. Chang, K. Lee, and K. Toutanova (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 4171–4186.

Dosovitskiy, A. et al. (2021). "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations (ICLR)*.

Ester, M., H.-P. Kriegel, J. Sander, and X. Xu (1996). "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise". In: *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 226–231.

Fawcett, T. (2006). "An Introduction to ROC Analysis". In: *Pattern Recognition Letters* 27.8, pp. 861–874.

Glorot, X. and Y. Bengio (2010). "Understanding the Difficulty of Training Deep Feedforward Neural Networks". In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256.

Goodfellow, I., Y. Bengio, and A. Courville (2016). *Deep Learning*. Cambridge, MA: MIT Press.

Goodfellow, I. et al. (2014). "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems 27*, pp. 2672–2680.

Harris, C. R., K. J. Millman, S. J. van der Walt, et al. (2020). "Array Programming with NumPy". In: *Nature* 585, pp. 357–362.

Hastie, T., R. Tibshirani, and J. Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer.

He, K., X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick (2022). "Masked Autoencoders Are Scalable Vision Learners". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009.

He, K., X. Zhang, S. Ren, and J. Sun (1915). "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034.

He, K., X. Zhang, S. Ren, and J. Sun (2016). "Deep Residual Learning for Image Recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.

Ho, J., A. Jain, and P. Abbeel (2020). "Denoising Diffusion Probabilistic Models". In: *Advances in Neural Information Processing Systems 33*, pp. 6840–6851.

Hochreiter, S. and J. Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780.

Hoffmann, J. et al. (2022). "Training Compute-Optimal Large Language Models". In: *arXiv preprint arXiv:2203.15556*.

Hornik, K., M. Stinchcombe, and H. White (1989). "Multilayer Feedforward Networks Are Universal Approximators". In: *Neural Networks* 2.5, pp. 359–366.

Hu, E. J. et al. (2022). "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*.

Jouppi, N. P., C. Young, N. Patil, et al. (2017). "In-Datacenter Performance Analysis of a Tensor Processing Unit". In: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–12.

Kaplan, J., S. McCandlish, T. Henighan, et al. (2020). "Scaling Laws for Neural Language Models". In: *arXiv preprint arXiv:2001.08361*.

Kingma, D. P. and J. Ba (2015). "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*, pp. 1–15.

Kingma, D. P. and M. Welling (2014). "Auto-Encoding Variational Bayes". In: *International Conference on Learning Representations (ICLR)*, pp. 1–9.

Krizhevsky, A., I. Sutskever, and G. E. Hinton (2012). "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems 25*, pp. 1097–1105.

LeCun, Y., Y. Bengio, and G. Hinton (2015). "Deep Learning". In: *Nature* 521.7553, pp. 436–444.

LeCun, Y., L. Bottou, Y. Bengio, and P. Haffner (1998). "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.

Lewis, P. et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems 33*, pp. 9459–9474.

Liu, F. T., K. M. Ting, and Z.-H. Zhou (2008). "Isolation Forest". In: *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pp. 413–422.

Loshchilov, I. and F. Hutter (2019). "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations (ICLR)*.

Maaten, L. van der and G. Hinton (2008). "Visualizing Data using t-SNE". In: *Journal of Machine Learning Research* 9, pp. 2579–2605.

MacQueen, J. (1967). "Some Methods for Classification and Analysis of Multivariate Observations". In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1, pp. 281–297.

McInnes, L., J. Healy, and J. Melville (2018). "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction". In: *arXiv preprint arXiv:1802.03426*.

McKinney, W. (2010). "Data Structures for Statistical Computing in Python". In: *Proceedings of the 9th Python in Science Conference (SciPy)*, pp. 56–61.

Mikolov, T., I. Sutskever, K. Chen, G. S. Corrado, and J. Dean (2013). "Distributed Representations of Words and Phrases and Their Compositionality". In: *Advances in Neural Information Processing Systems 26*, pp. 3111–3119.

Minsky, M. and S. Papert (1969). *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press.

Mitchell, T. M. (1997). *Machine Learning*. New York: McGraw-Hill.

Mnih, V., K. Kavukcuoglu, D. Silver, et al. (2015). "Human-level Control Through Deep Reinforcement Learning". In: *Nature* 518.7540, pp. 529–533.

Nair, V. and G. E. Hinton (2010). "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pp. 807–814.

Oord, A. van den, Y. Li, and O. Vinyals (2018). "Representation Learning with Contrastive Predictive Coding". In: *arXiv preprint arXiv:1807.03748*.

Ouyang, L., J. Wu, X. Jiang, et al. (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems 35*, pp. 27730–27744.

Patterson, D., J. Gonzalez, Q. Le, et al. (2021). "Carbon Emissions and Large Neural Network Training". In: *Advances in Neural Information Processing Systems 34*, pp. 13470–13481.

Pedregosa, F., G. Varoquaux, A. Gramfort, et al. (2011). "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12, pp. 2825–2830.

Radford, A., K. Narasimhan, T. Salimans, and I. Sutskever (2018). "Improving Language Understanding by Generative Pre-Training". In: *OpenAI Blog*.

Rafailov, R., A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn (2023). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". In: *Advances in Neural Information Processing Systems 36*.

Rosenblatt, F. (1958). "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6, pp. 386–408.

Rumelhart, D. E., G. E. Hinton, and R. J. Williams (1986). "Learning Representations by Back-Propagating Errors". In: *Nature* 323.6088, pp. 533–536.

Russell, S. J. and P. Norvig (2021). *Artificial Intelligence: A Modern Approach*. 4th ed. Hoboken, NJ: Pearson.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). "Proximal Policy Optimization Algorithms". In.

Searle, J. R. (1980). "Minds, Brains, and Programs". In: *Behavioral and Brain Sciences* 3.3, pp. 417–424.

Silver, D., A. Huang, C. J. Maddison, et al. (2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587, pp. 484–489.

Strang, G. (2019). *Linear Algebra and Learning from Data*. Wellesley, MA: Wellesley–Cambridge Press.

Strubell, E., A. Ganesh, and A. McCallum (2019). "Energy and Policy Considerations for Deep Learning in NLP". In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3645–3650.

Su, J., M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu (2024). "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *Neurocomputing* 568, p. 127063.

Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press.

Turing, A. M. (1950). "Computing Machinery and Intelligence". In: *Mind* 59.236, pp. 433–460.

VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol, CA: O'Reilly Media.

Vaswani, A. et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30*, pp. 5998–6008.

Vincent, P., H. Larochelle, Y. Bengio, and P.-A. Manzagol (2008). "Extracting and Composing Robust Features with Denoising Autoencoders". In: *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pp. 1096–1103.

Wasserman, L. (2004). *All of Statistics: A Concise Course in Statistical Inference*. New York: Springer.

Watkins, C. J. C. H. and P. Dayan (1992). "Q-Learning". In: *Machine Learning* 8.3-4, pp. 279–292.

Wolf, T. et al. (2020). “Transformers: State-of-the-Art Natural Language Processing”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45.

Zhang, A., Z. C. Lipton, M. Li, and A. J. Smola (2023). *Dive into Deep Learning*. Cambridge: Cambridge University Press.

— End of document —